

České vysoké učení technické v Praze
Fakulta jaderná a fyzikálně inženýrská

Katedra softwarového inženýrství
Program: Aplikace informatiky v přírodních vědách
Specializace: –



Algoritmy strojového učení pro
adaptaci autonomních agentů v 3D
akční hře

Machine Learning Algorithms for
Adapting Autonomous Agents in a
3D Action Game

DIPLOMOVÁ PRÁCE

Vypracoval: Bc. Martin Johec
Vedoucí práce: Ing. Josef Nový, Ph.D.
Rok: 2026

I. OSOBNÍ A STUDIJNÍ ÚDAJE

Příjmení: **Jochec** Jméno: **Martin** Osobní číslo: **502714**
Fakulta/ústav: **Fakulta jaderná a fyzikálně inženýrská**
Zadávající katedra/ústav: **Katedra softwarového inženýrství**
Studijní program: **Aplikace informatiky v přírodních vědách**

II. ÚDAJE K DIPLOMOVÉ PRÁCI

Název diplomové práce:

Algoritmy strojového učení pro adaptaci autonomních agentů v 3D akční hře

Název diplomové práce anglicky:

Machine Learning Algorithms for Adapting Autonomous Agents in a 3D Action Game

Jméno a pracoviště vedoucí(ho) diplomové práce:

Ing. Josef Nový, Ph.D. Katedra softwarového inženýrství FJFI Děčín

Jméno a pracoviště druhé(ho) vedoucí(ho) nebo konzultanta(ky) diplomové práce:

Datum zadání diplomové práce: **29.10.2024**

Termín odevzdání diplomové práce: **05.01.2026**

Platnost zadání diplomové práce: **28.10.2026**

Digitálně podepsal(a)
Radek Fučík
Datum: 29.10.2025
17:47:50
ID: 305323

podpis vedoucí(ho) ústavu/katedry

Digitálně podepsal(a)
Milan Krbálek
Datum: 30.10.2025
09:19:52
ID: 40556

podpis proděkana(ky) z pověření děkana(ky)

III. PŘEVZETÍ ZADÁNÍ

Diplomant bere na vědomí, že je povinen vypracovat diplomovou práci samostatně, bez cizí pomoci, s výjimkou poskytnutých konzultací. Seznam použité literatury, jiných pramenů a jmen konzultantů je třeba uvést v diplomové práci.

04.11.2025
Datum převzetí zadání

Bc. Jochec Martin
Podpis studenta

PROHLÁŠENÍ

Já, níže podepsaný

Příjmení, jméno studenta: Johec Martin
Osobní číslo: 502714
Název programu: Aplikace informatiky v přírodních vědách

prohlašuji, že jsem diplomovou práci s názvem

Algoritmy strojového učení pro adaptaci autonomních agentů v 3D akční hře

vypracoval samostatně a uvedl veškeré použité informační zdroje v souladu s Metodickým pokynem o dodržování etických principů při přípravě vysokoškolských závěrečných prací a Rámcovými pravidly používání umělé inteligence na ČVUT pro studijní a pedagogické účely v Bc a NM studiu.

Prohlašuji, že jsem v průběhu příprav a psaní závěrečné práce použil nástroje umělé inteligence. Vygenerovaný obsah jsem ověřil. Stvrzuji, že jsem si vědom, že za obsah závěrečné práce plně zodpovídám.

V Praze dne 16.11.2025

Bc. Martin Johec

.....
podpis studenta

Poděkování

Děkuji Ing. Josefu Novému, Ph.D., za vedení mé diplomové práce a za případnou pomoc při její tvorbě.

Bc. Martin Johec

Název práce:

Algoritmy strojového učení pro adaptaci autonomních agentů v 3D akční hře

Autor: Bc. Martin Johech

Studijní program: Aplikace informatiky v přírodních vědách

Specializace: –

Druh práce: Diplomová práce

Vedoucí práce: Ing. Josef Nový, Ph.D.

Katedra softwarového inženýrství, Fakulta jaderná a fyzikálně inženýrská, České vysoké učení technické v Praze

Konzultant: –

Abstrakt: Tato diplomová práce se zabývá algoritmy strojového učení pro adaptaci autonomního agenta na chování uživatele ve 3D akční hře. Hlavním cílem je vytvořit jednoduchého autonomního agenta, který dokáže přizpůsobit své chování i vybavení protihráči. Práce představuje existující herní agenty, jako jsou AlphaStar, OpenAI Five a Voyager, a popisuje některé algoritmy a techniky používané při jejich vývoji – například Proximal Policy Optimization nebo Self-Play. Dále se zaměřuje na použité technologie, například Unreal Engine či NevarokML, a na samotný návrh, který je s jejich pomocí realizován. Závěrem jsou výsledky experimentu vyhodnoceny.

Klíčová slova: hry, umělá inteligence, adaptace, autonomní agent, strojové učení

Title:

Machine Learning Algorithms for Adapting Autonomous Agents in a 3D Action Game

Author: Bc. Martin Johech

Abstract: This thesis focuses on machine learning algorithms for adapting an autonomous agent to user behavior in a 3D action game. The main objective is to create a simple autonomous agent capable of adjusting its behavior and equipment in response to the player. The work introduces existing game agents such as AlphaStar, OpenAI Five, and Voyager, and describes some of the algorithms and techniques used in their development – for example, Proximal Policy Optimization and Self-Play. It then focuses on the technologies used, such as Unreal Engine and NevarokML, and the design that is implemented using them. Finally, the results of the experiment are evaluated.

Key words: games, artificial intelligence, adaptation, autonomous agent, machine learning

Obsah

Úvod	9
1 AI agenti ve hrách a jejich algoritmy a techniky	11
1.1 Strojové učení a algoritmy zpětnovazebního učení	11
1.1.1 Proximal Policy Optimization (PPO)	12
1.1.2 Deep Q-Network	15
1.1.3 Self-Play	18
1.1.4 Population Based Training	21
1.2 AI agenti ve hrách	24
1.2.1 AlphaStar	24
1.2.2 OpenAI Five	26
1.2.3 Voyager	28
2 Použité technologie a návrh hry	31
2.1 Použité technologie	31
2.1.1 Unreal Engine	31
2.1.2 Visual Studio	33
2.1.3 NevarokML	34
2.2 Návrh hry	36
2.2.1 Návrh prostředí hry	36
2.2.2 Návrh agenta	37
3 Implementace hry a agenta, vylepšení jeho algoritmu a porovnání verzí	39
3.1 Základní logika hry	39
3.1.1 Hráčská postava	39
3.1.2 Zbraňový systém	41
3.1.3 Logika řízení herních kol	42
3.1.4 Systém sbírání předmětů	42
3.1.5 Krabice s vybavením	43
3.1.6 Herní rozhraní (HUD)	43
3.1.7 Herní prostředí	44
3.1.8 Nepřátelské postavy	44
3.2 Autonomní agent	45
3.2.1 První verze agenta	45
3.2.2 Vyhodnocení první verze agenta	54
3.2.3 Druhá verze agenta	58

3.2.4	Vyhodnocení druhé verze agenta	64
3.2.5	Finální verze agenta	68
3.2.6	Vyhodnocení finální verze agenta	69
3.2.7	Shrnutí všech verzí	75
Závěr		77
Literatura		79
Přílohy		82
A Jak zapnout program		83

Úvod

Tato diplomová práce se zabývá algoritmy strojového učení pro adaptaci autonomního agenta na chování uživatele ve 3D akční hře. Hlavním cílem je navrhnout jednoduchého autonomního agenta, který se pomocí strojového učení a změny vybavení dokáže přizpůsobit protivníkovi a porazit ho.

První kapitola obsahuje dvě podkapitoly. První se věnuje vybraným technikám a algoritmům strojového učení, jako jsou Proximal Policy Optimization (PPO), Deep Q-Network (DQN), a techniky Self-Play a Population Based Training (PBT). Druhá podkapitola popisuje existující herní agenty, konkrétně AlphaStar, OpenAI Five a Voyager, včetně přehledu jejich fungování a způsobu implementace.

Druhá kapitola se zaměřuje na použité technologie a návrh samotné hry. První podkapitola podrobně popisuje využití Unreal Engine, Visual Studio a pluginu Nevarok ML, který rozšiřuje Unreal Engine o funkcionalitu pro trénink a aplikaci modelů strojového učení. Druhá podkapitola se věnuje návrhu herního prostředí a autonomního agenta.

Třetí kapitola popisuje implementaci hry a agenta. První část se zaměřuje na herní logiku, která poskytuje prostředí pro aplikování agenta. Druhá část se věnuje implementaci samotné logiky agenta, pokusům o vylepšení jeho logiky a porovnání a hodnocení různých verzí.

Závěr shrnuje úvahy autora a dosažené výsledky — zda se agent adaptuje podle očekávání, jaký vliv mají jednotlivá vylepšení a jak si různé verze modelu vedou ve vzájemném porovnání.

Kapitola 1

AI agenti ve hrách a jejich algoritmy a techniky

1.1 Strojové učení a algoritmy zpětnovazebního učení

Strojové učení je samostatná oblast umělé inteligence, která se zabývá vývojem metod umožňujících počítačovým systémům automaticky zlepšovat své chování na základě zkušeností bez nutnosti explicitního programování všech rozhodovacích pravidel. Takové systémy se učí přímo z příkladů nebo interakcí s prostředím, identifikují vzory a vytvářejí modely, které generalizují i na nové situace, jež nebyly přímo obsaženy v tréninkových datech.

Takový přístup je zásadní zejména tam, kde ruční definice pravidel nevede k dobrému výsledku – například při řízení autonomních agentů v komplexních, nelineárních a stochastických prostředích nebo při adaptivním rozhodování v dynamických situacích, které nelze přesně popsat jednou sadou pravidel.

Obecně rozlišujeme několik hlavních paradigmat strojového učení, z nichž každé má své typické úlohy a metody:

1. **Učení s učitelem (Supervised Learning)** – modely se učí na označených datech, kde jsou vstupy i správné výstupy známy, a cílem je minimalizovat rozdíl mezi predikcemi modelu a skutečnými výstupy.
2. **Učení bez učitele (Unsupervised Learning)** – model hledá strukturu nebo vzory v neoznačených datech bez explicitně definovaných výstupů.
3. **Částečně řízené učení (Semi-Supervised Learning)** – kombinuje omezené množství označených dat s velkým množstvím neoznačených dat pro efektivnější učení.
4. **Zpětnovazební učení (Reinforcement Learning, RL)** – agent se učí sekvenci rozhodnutí pomocí interakcí s prostředím, kdy dostává odměny podle kvality svých akcí [22].

V této práci se zaměřujeme výhradně na zpětnovazební učení. RL se liší od ostatních paradigmat především tím, že agent nepracuje pouze s pevnými daty, ale aktivně ovlivňuje svět kolem sebe, sleduje výsledky svých rozhodnutí a učí se na základě toho. Tento způsob učení se formálně modeluje pomocí *Markovských rozhodovacích procesů (MDP)* – matematického rámce pro rozhodování v prostředí s náhodnými přechody.

MDP je definován jako pětice $(\mathcal{S}, \mathcal{A}, P, R, \gamma)$, kde:

- $\mathcal{S}$ je množina možných stavů prostředí,
- $\mathcal{A}$ je množina možných akcí, které může agent vykonat,
- $P(s' | s, a)$ je přechodová pravděpodobnost, tj. pravděpodobnost, že agent skončí ve stavu s' poté, co ve stavu s zvolí akci a ,
- $R(s, a)$ je okamžitá odměna přidělená agentovi za provedení akce a ve stavu s ,
- $\gamma \in [0, 1]$ je diskontní faktor, který určuje, jak moc si agent cení budoucích odměn vůči těm bezprostředním [22].

Agent volí akce podle tzv. *politiky* $\pi(a | s)$ – funkce, která pro každý stav s přiřazuje pravděpodobnost výběru akce a . RL algoritmy se snaží nalézt optimální politiku π^* , která maximalizuje očekávanou sumu diskontovaných odměn za dlouhé časové horizonty.

1.1.1 Proximal Policy Optimization (PPO)

Proximal Policy Optimization (PPO) je jeden z nejrozšířenějších algoritmů hlubokého zpětnovazebního učení, který byl navržen výzkumníky OpenAI jako efektivní a stabilní alternativa k dřívějším policy gradient a trust region metodám [8][10][33]. PPO spojuje relativní jednoduchost implementace s vysokou stabilitou učení a efektivním využitím nasbíraných dat.

Na rozdíl od hodnotových metod, jako je Q-learning [14][22], které se snaží nejprve odhadnout hodnoty stavů či stav-akce a teprve poté z nich generovat politiku, PPO přímo optimalizuje politiku parametrizovanou parametry θ (např. vahami neuronové sítě). Cílem je nalézt taková parametry θ , která vedou k maximalizaci dlouhodobé očekávané odměny.

Klíčovým problémem u jednodušších policy gradient metod je to, že jednotlivé aktualizace mohou způsobit příliš velké změny v politice, což může destabilizovat proces učení. Metody jako TRPO (Trust Region Policy Optimization) řeší tento problém zavedením explicitních omezení na velikost kroku při aktualizaci politiky, ale tyto přístupy jsou výpočetně náročné a komplikované na implementaci [9]. PPO tento problém řeší pomocí tzv. *clipped surrogate objective*, která udržuje aktualizace politiky v kontrolovaném intervalu bez složitých optimalizačních podproblémů.

Nejprve se zavádí poměr pravděpodobností mezi novou a starou politikou:

$$r_t(\theta) = \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_{\text{old}}}(a_t \mid s_t)}, \quad (1.1)$$

kde π_θ je nová politika, $\pi_{\theta_{\text{old}}}$ je politika před aktualizací, s_t je aktuální stav a a_t akce zvolená ve stavu s_t . Tento poměr udává, jak moc se změnila pravděpodobnost výběru konkrétní akce mezi starou a novou politikou.

Poté se definuje *oříznutý surrogate loss*:

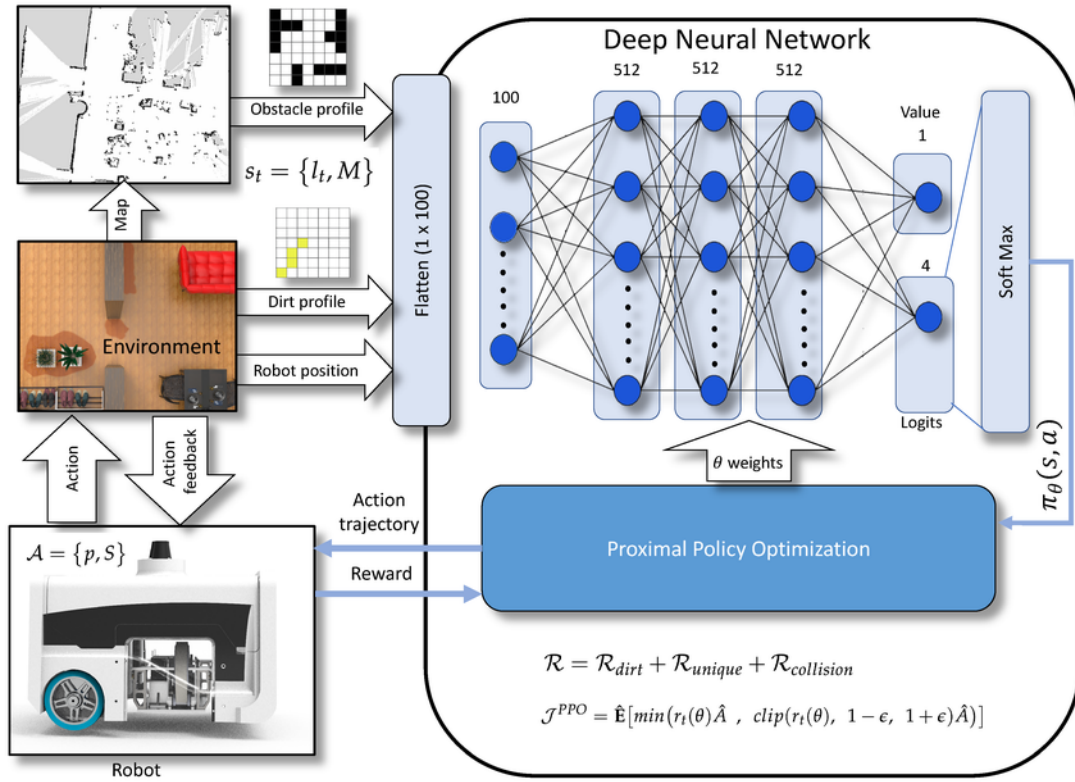
$$L^{\text{CLIP}}(\theta) = \hat{\mathbb{E}}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip} \left(r_t(\theta), 1 - \epsilon, 1 + \epsilon \right) \hat{A}_t \right) \right], \quad (1.2)$$

kde:

- $\hat{A}_t$ je odhad tzv. *advantage funkce*, která udává, jak moc je akce a_t ve stavu s_t lepší oproti průměrnému chování,
- ϵ je hyperparametr určující rozsah, ve kterém změna politiky není penalizována,
- clip ořízne hodnotu $r_t(\theta)$ do intervalu $[1 - \epsilon, 1 + \epsilon]$, čímž se minimalizují příliš velké změny politiky [10].

Výraz (1.2) říká, že pokud je změna politiky menší než povolené pásmo dané ϵ , ponechá se původní hodnota. Pokud je změna větší, bere se hodnota oříznutá na hranici přijatelného rozsahu. Tento postup zabraňuje vykročení politiky do oblastí, kde by mohla být nová politika nestabilní nebo výrazně horší než ta původní.

Obrázek 1.1 vizualizuje architekturu PPO a jeho interakci s prostředím:



Obrázek 1.1: Vizualizace architektury algoritmu PPO. Agent je ovlivňován prostředím, zatímco dvě neuronové sítě — actor a critic — slouží k výběru akcí a odhadu hodnoty stavu. Politika se aktualizuje pomocí oříznutého objektivu (clipped objective) a výhody se počítají pomocí metody Generalized Advantage Estimation (GAE).[28]

PPO lze dále rozšířit o dodatečné složky, které stabilizují učení a podporují explorační prostředí:

$$L^{PPO}(\theta) = \mathbb{E}_t [L_t^{\text{CLIP}}(\theta) - c_1 L_t^{\text{VF}}(\theta) + c_2 \mathcal{S}(\pi_\theta, s_t)] , \quad (1.3)$$

kde L^{VF} je ztráta hodnotové funkce, $\mathcal{S}(\pi_\theta, s_t)$ entropie politiky ve stavu s_t a c_1, c_2 jsou váhové koeficienty určující jejich vliv na celkový loss [10].

1.1.2 Deep Q-Network

Algoritmus Deep Q-Network (DQN)[22][23][33][24] představuje metodu zpětnovazebního učení, která kombinuje klasický Q-learning[2] s hlubokými neuronovými sítěmi pro aproximaci akčně-hodnotové funkce $Q(s, a; \theta)$. Tato funkce udává očekávanou kumulovanou diskontovanou odměnu, kterou agent získá, pokud ve stavu s provede akci a a poté pokračuje optimálním chováním.

Teoretickým základem DQN je Bellmanova optimální rovnice:

$$Q^*(s_t, a_t) = \mathbb{E} \left[r_{t+1} + \gamma \max_{a'} Q^*(s_{t+1}, a') \mid s_t, a_t \right], \quad (1.4)$$

kde s_t je aktuální stav, a_t zvolená akce, r_{t+1} okamžitá odměna, s_{t+1} následující stav a $\gamma \in [0, 1]$ diskontní faktor, který určuje relativní váhu budoucích odměn oproti bezprostředním.

V klasickém Q-learningu se toto řeší pomocí aktualizací pravidla:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left(r_{t+1} + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t) \right), \quad (1.5)$$

kde α je rychlost učení. Pro prostory se stovkami či tisíci stavů by taková tabulka byla nepraktická nebo nemožná. DQN proto využívá neuronovou síť parametrizovanou θ , která na vstup přijímá reprezentaci stavu s a na výstup generuje **vektor Q-hodnot pro všechny akce**:

$$Q(s, a; \theta) \approx Q^*(s, a). \quad (1.6)$$

Trénink DQN probíhá minimalizací **časového rozdílu (TD error)** mezi odhadovanou Q-hodnotou a cílovou hodnotou odvozenou z Bellmanovy rovnice. Kódování těchto odhadů a stabilizace učení používá dvě hlavní techniky:

- *Přehrávací paměť* ($\mathcal{D}$) – ukládá přechody $(s_t, a_t, r_{t+1}, s_{t+1})$ a umožňuje náhodné vzorkování mini-batchů. Tím se přerušuje autokorelace mezi po sobě následujícími vzorky a zvyšuje se rozmanitost tréninkových dat.
- *Cílová síť* – označovaná $Q(s, a; \theta^-)$, což je periodicky aktualizovaná kopie online sítě. Cílová síť poskytuje stabilní referenční hodnoty během učení.

Pro výpočet cílové hodnoty a ztrátové funkce DQN se používají následující výrazy. Nejprve definujeme cílovou hodnotu y_t :

$$y_t = r_{t+1} + \gamma \max_{a'} Q(s_{t+1}, a'; \theta^-). \quad (1.7)$$

Tento výraz vyjadřuje tzv. *temporal difference target*. Skládá se ze dvou částí:

- *Okamžitá odměna* r_{t+1} : okamžitý zisk, který agent získá provedením akce a_t ve stavu s_t .
- *Odhad budoucích odměn* $\gamma \max_{a'} Q(s_{t+1}, a'; \theta^-)$: diskontovaná hodnota nejlepší možné akce v následujícím stavu s_{t+1} , podle cílové sítě s parametry θ^- . Tento člen reprezentuje odhad kumulativní budoucí odměny.

Hyperparametr γ je diskontní faktor, který určuje relativní váhu budoucích odměn vůči bezprostřední odměně. V praxi se hodnoty γ blíží 1 v úlohách, kde je důležitý dlouhodobý výsledek, a menší γ se používá tam, kde jsou odměny krátkodobé.

Následuje výpočet ztrátové funkce $L_t(\theta)$:

$$L_t(\theta) = \mathbb{E}_{(s,a,r,s') \sim \mathcal{D}} \left[(y_t - Q(s_t, a_t; \theta))^2 \right]. \quad (1.8)$$

Tato funkce vyjadřuje **střední kvadratickou chybu (mean squared error)** mezi:

- *aktuálním odhadem* hodnoty Q získaným z online sítě $Q(s_t, a_t; \theta)$, kterou trénujeme,
- *cílovou hodnotou* y_t , která využívá cílovou síť a okamžitou odměnu.

Minimalizací této chyby učíme síť tak, aby její predikce Q -hodnot co nejlépe odpovídaly cílovým Bellmanovým odhadům. Důležité je, že cílové hodnoty se během tréninku **nepřenastavují okamžitě**, ale fixují se na delší interval pomocí zmíněné cílové sítě $Q(s, a; \theta^-)$. To zásadně pomáhá stabilizovat učení a zabránit oscilacím nebo divergenci při aktualizaci parametrů.

Po definici loss funkce následuje **aktualizace parametrů** pomocí gradientního sestupu:

$$\theta \leftarrow \theta - \alpha \nabla_{\theta} L_t(\theta), \quad (1.9)$$

kde:

- α je rychlost učení (learning rate), která určuje, jak velký krok do směru negativního gradientu uděláme v každé aktualizaci,
- $\nabla_{\theta} L_t(\theta)$ je gradient loss funkce vůči parametrům sítě θ . Tento gradient udává, jak změna každého parametru ovlivní chybu.

Gradientní sestup směřuje parametry sítě tak, aby se predikovaná Q-hodnota $Q(s_t, a_t; \theta)$ blížila cílovému odhadu y_t . V důsledku toho síť postupně zlepšuje své odhady Q-hodnot pro různé stav-akce páry.

Poslední rovnice vyjadřuje **synchronizaci parametrů cílové sítě** s hlavní sítí:

$$\theta^- \leftarrow \theta. \quad (1.10)$$

Tento krok se neprovádí při každé aktualizaci, ale periodicky (např. každých K tréninkových kroků). V praxi se tím zajistí, že cílová síť zůstane dostatečně „fixní“ po určitou dobu, což zabraňuje tomu, aby se cílové hodnoty měnily příliš rychle a destabilizovaly tréninkový proces. Bez této separace by totiž síť během učení upravovala své parametry tak, že by se sama učila na neustále se měnící cílové funkci — což má tendenci vést k oscilacím nebo divergenci.

Stručně řečeno:

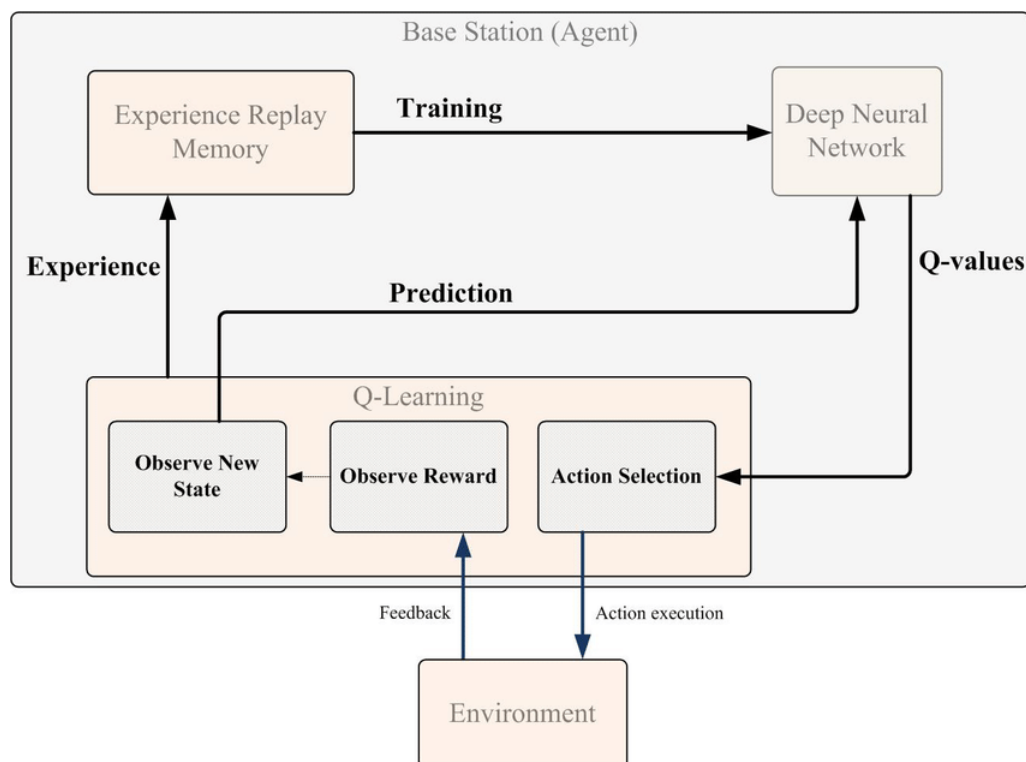
- *Loss funkce* měří, jak moc se predikce odchyluje od cílové hodnoty na základě Bellmanovy rovnice.
- *Gradientní aktualizace* upravuje váhy tak, aby se tato odchylka minimalizovala.
- *Target network* odděluje výpočet cílových Q-hodnot od parametrů, které se právě učí, čímž stabilizuje trénink.

Tréninkový cyklus DQN tedy typicky probíhá následovně:

1. Sledování aktuálního stavu s_t .
2. Výběr akce a_t pomocí ϵ -greedy politiky.
3. Provedení akce, získání odměny r_{t+1} a nového stavu s_{t+1} .
4. Uložení přechodu do paměti $\mathcal{D}$.
5. Náhodné vzorkování mini-batchů dat z $\mathcal{D}$.
6. Výpočet cílové hodnoty y_t a minimalizace TD chyby.
7. Periodická aktualizace parametrů cílové sítě ($\theta^- \leftarrow \theta$).

Algoritmus DQN je citlivý na nestabilitu a divergenci kvůli kombinaci nelineární aproximace neuronovou sítí a bootstrapovaných aktualizací. Přehrávací paměť redukuje autokorelaci dat a zvyšuje efektivitu využití zkušeností, zatímco cílová síť poskytuje stabilnější referenční hodnoty. Pokročilé varianty, jako jsou *Double DQN*, *Dueling DQN* nebo *Prioritized Experience Replay*, rozšiřují základní metodu s cílem zlepšit konvergenci, stabilitu a efektivitu učení v komplexních prostředích.

Tyto dva algoritmy byly již zmíněny ve výzkumné práci[33], kde tato diplomová práce na ni navazuje. Teď budou zmíněny dvě trénovací techniky, Self-Play a Population Based Training.



Obrázek 1.2: Příklad architektury Deep Q-Learning[31]

1.1.3 Self-Play

Self-play[17] je technika tréninku v oblasti zpětnovazebního učení, která umožňuje agentům zlepšovat své strategie prostřednictvím interakcí s jinými instancemi sebe sama – typicky s kopiemi, historickými verzemi nebo současnými protivníky založenými na stejné struktuře politiky. Cílem self-play je generovat adaptivní tréninková data bez nutnosti externích protivníků či lidského dozoru, čímž se agent učí robustní strategii na základě postupného zvyšování úrovně soupeře během tréninku.

Na rozdíl od klasického zpětnovazebního učení, kde agent interaguje se statickým prostředím, self-play funguje v rámci dynamického prostředí, které vytváří další agentní entity, jejichž chování se v čase vyvíjí a reaguje na aktuální strategii. To vede k učení, která mohou překročit jednoduché optimalizační úlohy a dosahovat hlubší strategické adaptace, zejména v úlohách s konkurenční povahou.

Základní forma self-play, často označovaná jako „vanilla self-play“, spočívá v tom, že agent opakovaně hraje proti své vlastní nejnovější verzi. Tato metoda funguje efektivně v prostředích, kde lze strategie řadit podle jejich kvality (transitivní hry), protože agent postupně zvyšuje úroveň soupeře sám proti sobě. V prostředích s ne-transitivní dynamikou může tato jednoduchá forma self-play vést k cyklickému chování a suboptimálním strategiím.

Aby se tyto nevýhody zmírnily, vznikají různé varianty:

- *Fiktivní self-play* rozšiřuje trénink tým, že agent hraje proti průměru všech svých předchozích verzí politiky. Tím se zvyšuje robustnost učení a snižuje riziko zacyklení v suboptimálních strategiích.
- *Prioritizovaná fiktivní self-play* používá vážený výběr soupeřů, kde se preferují ty politiky, proti kterým si agent vede méně dobře – tedy trénuje intenzivněji vůči slabým stránkám vlastní strategie.
- *δ -uniform self-play* zavádí hyperparametr δ , který určuje, jaká část historické populace politik se použije jako soupeři. Nižší δ zahrnuje širší spektrum historických politik, zatímco vyšší δ dává přednost nedávno generovaným verzím.

Self-play se často integruje s tzv. *population-based* přístupy. Namísto jednoho agenta existuje populace politik Π , která se v čase rozšiřuje o nové verze. Každá politika se trénuje vůči soupeřům vybraným z populace na základě určité metastrategie Σ , často reprezentované interakční maticí nebo distribuční strategií. Tento proces vytváří bohatý a adaptivní prostor pro evoluci strategií a vzájemnou adaptaci politik.

Základní smyčka self-play s populací obvykle probíhá následovně:

1. Nové politiky jsou inicializovány, často na základě posledních iterací existujících politik.
2. Každá politika se trénuje proti soupeřům vybraným podle metastrategie Σ .
3. Nově natrénované politiky se přidávají zpět do populace a proces se opakuje.

Z hlediska výpočetní efektivity se rozlišují varianty jako:

- *lazy initialization*, kdy se politiky inicializují až v okamžiku potřeby,
- *immediate initialization*, kdy se populace vytvoří celá najednou na začátku.

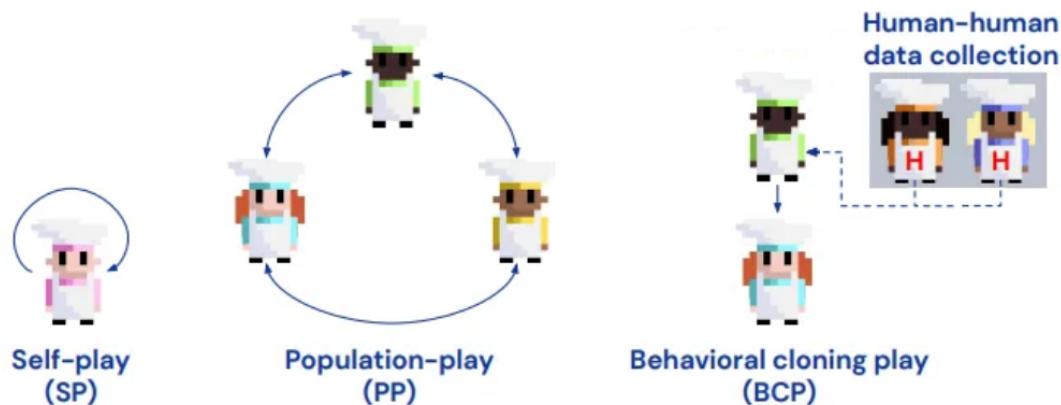
Dále se rozlišuje:

- *standardní trénink*, kde se v každé iteraci trénuje jeden agent,
- *průběžný trénink*, kdy se všechny politiky trénují paralelně.

Zejména v kontextu větších populací a her s komplexní strategií hraje důležitou roli tzv. PSRO (*Policy-Space Response Oracles*). PSRO rozšiřuje klasické self-play tak, že pracuje s populací politik a iterativně generuje nejlepší odpovědi na směs politik populace, čímž se přibližuje rovnováze typu Nashovy rovnováhy. To je důležité zejména v nestranově tranzitivních nebo multi-agentních hrách, kde jednoduché sebehráčské strategie nemusí konvergovat ke stabilnímu optimu.

Self-play nachází využití v konkurenčních a multi-agentních prostředích — klasickými příklady jsou hry jako Go, šachy, StarCraft II nebo Dota 2 — a také v jiných oblastech, například v některých aplikacích robotiky, kde dynamické protinavádění podporuje robustní strategie bez nutnosti lidského tréninkového dozoru.

Obrázek 1.3 ilustruje rozdíly mezi přístupy „self-play“ (SP), „population-play“ (PP) a „behavioral cloning play“ (BCP). Druhý ze zmíněných přístupů využívá širší populace politik, zatímco BCP je založen na učení z lidských demonstračních dat, což je odlišné od čistě self-play přístupů, které generují data prostřednictvím interakcí agentů mezi sebou.



Obrázek 1.3: Srovnání tří přístupů k tréninku agentů: „self-play“ (SP), kde agent hraje sám proti sobě; „population-play“ (PP), kde interaguje s různými agenty v populaci; a „behavioral cloning play“ (BCP), kde se učení zakládá na lidských demonstračních datech. Upraveno podle zdroje[29]

1.1.4 Population Based Training

Population-Based Training (PBT)[5] je technika automatického ladění hyperparametrů, která propojuje samotný trénink neuronových sítí s jejich optimalizací tak, aby se váhy sítě a její hyperparametry upravovaly současně během jednoho běhu učení. PBT vzniká jako reakce na omezení tradičních metod ladění, jako je grid search nebo Bayesian optimization, které často vyžadují mnoho samostatných tréninků s fixními konfiguracemi a mohou být časově i výpočetně náročné.

V PBT se nevyhledává pouze jedna nejlepší sada hyperparametrů, ale algoritmus se snaží **automaticky objevit jejich optimální plán (schedule)** během tréninku samotného modelu — díky tomu se může přizpůsobovat různým fázím učení, kdy různé hodnoty hyperparametrů jsou vhodné v různých časech tréninku.

Na začátku PBT inicializuje populaci modelů, přičemž každá instance populace má:

- vlastní váhy (parametry neuronové sítě),
- vlastní konfiguraci hyperparametrů (např. learning rate, entropická penalizace, délka unrollu apod.).

Každý model se trénuje paralelně a nezávisle pomocí běžných optimalizačních metod (např. SGD nebo Adam). V pravidelných intervalech se populace vyhodnocuje a provádí tzv. *exploit-and-explore* fáze, které spojují principy evolučních algoritmů s gradientním učením.

Exploit znamená, že modely s horším výkonem přebírají váhy a/nebo hyperparametry od modelů s lepším výkonem. Tato operace je analogická selekci v genetických algoritmech — silnější jedinci „přežijí“ a jejich parametry se šíří do dalších populací. *Explore* znamená, že tyto převzaté hyperparametry jsou **lehce modifikovány** (např. vynásobením faktorem 1,2 nebo 0,8), aby populace prozkoumala nové konfigurace. Tím se neustále testují různé kombinace nastavení během tréninku a umožňuje se adaptivní ladění hyperparametrů bez nutnosti opakovaných běhů tréninku od začátku.

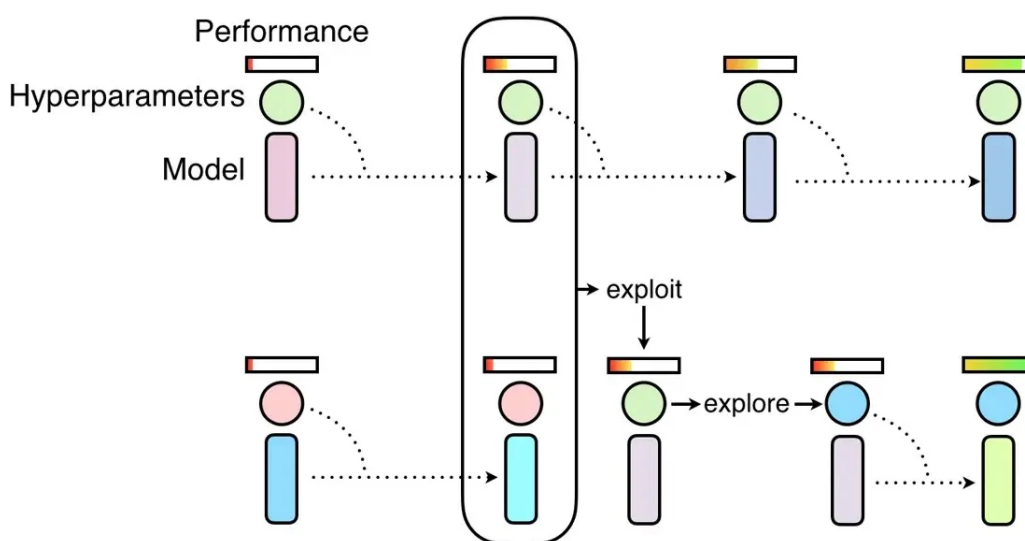
Proces lze shrnout následovně:

1. Každý model ve své instanci trénuje standardní metodou s aktuální konfigurací hyperparametrů.
2. Po určité době nebo podle metriky se vyhodnocuje výkon každého modelu.
3. Pokud je model označen jako „ready“, probíhá fáze „exploit“ – model si kopíruje váhy a hyperparametry od výkonnějších modelů v populaci.
4. Následuje fáze „explore“, při které se hyperparametry lehce mění (například násobky, malé perturbace nebo náhodné přegenerování).
5. Trénink pokračuje s nově nastavenými vahami a hyperparametry a celý cyklus se opakuje.

Tento způsob optimalizace kombinuje směry gradientního učení a evolučního vyhledávání tak, že:

- zajišťuje **adaptivní změnu hyperparametrů během učení**,
- eliminuje potřebu řešit tréninky s fixními konfiguracemi opakovaně,
- dovoluje paralelní škálování na heterogenních výpočetních systémech bez centrální koordinace.

Obrázek 1.4 ilustruje, jak si modely v rámci populace sdílejí váhy a mění hyperparametry na základě výkonnosti:



Obrázek 1.4: Population-Based Training neuronových sítí začíná jako náhodné prohledávání, ale v průběhu tréninku umožňuje jednotlivým modelům využít dílčí výsledky ostatních (fáze „exploit“) a zároveň zkoušet nové kombinace hyperparametrů (fáze „explore“).[30]

Klíčové principy PBT mohou být interpretovány jako hybridní systém:

- *výběr modelů* podobný truncation nebo turnajové selekci v evolučních metodách,
- *perturbace hyperparametrů* jako jednoduchý heuristický způsob prozkoumání prostoru hyperparametrů během běhu tréninku,
- *asynchronní aktualizace*, která eliminuje nutnost synchronizace všech modelů.

Tato technika byla v původní práci demonstrována na úlohách hlubokého zpětnovazebního učení, kde vedla nejen k rychlejší konvergenci, ale i k vyššímu konečnému výkonu modelů při optimalizaci různých hyperparametrů. PBT také nachází uplatnění v dalších oblastech, jako je strojové překládání nebo trénink generativních modelů, kde automaticky objevuje sekvence hyperparametrů, které překonávají ručně laděné konfigurace.

Důležitou výhodou PBT je jeho **online charakter** — ladění hyperparametrů probíhá současně s tréninkem modelů, což je efektivnější nejen časově, ale i z hlediska výpočetních zdrojů, protože se eliminuje potřeba trénovat modely od začátku po každé úpravě hyperparametrů. Tento aspekt výrazně odliší PBT od tradičních metod ladění, které obvykle vyžadují dokončení jednoho tréninkového běhu před dalším experimentem.

Celkově lze PBT vnímat jako techniku, která do procesu učení vnáší adaptivitu, sdílení informací mezi modely a evoluční principy, což podporuje robustnější a efektivnější optimalizaci ve srovnání s pevně danými metodami ladění.

1.2 AI agenti ve hrách

1.2.1 AlphaStar

AlphaStar[12] je agent umělé inteligence (AI) vyvíjený společností DeepMind pro hraní hry StarCraft II[16], což je komplexní a dynamická online strategie v reálném čase s částečnou pozorovatelností, rozsáhlým akčním prostorem a dlouhodobým plánováním. Hra StarCraft II je považována za jeden z nejnáročnějších testovacích prostorů pro vývoj inteligentních systémů, protože kombinuje strategické a taktické rozhodování se zpracováním neurčitých informací a interakcí více agentů.

AlphaStar byla veřejnosti představena v lednu 2019 a již v srpnu téhož roku dosáhla hodnosti „Grandmaster“, což je nejvyšší úroveň na oficiální žebříčkové lize StarCraft II a znamená, že se agent řadí mezi přibližně 0,2 % nejlepších lidských hráčů. Tento úspěch představuje významný milník nejen v oblasti herní AI, ale i ve vývoji komplexních autonomních systémů, které zvládají vysokodimenzionální rozhodovací úlohy.

StarCraft II jako prostředí klade na agenta řadu výzev, které nejsou přítomné v jednodušších hrách nebo simulacích: **časové omezení, skryté informace (fog of war), rozsáhlý akční prostor, dynamické kombinace jednotek a strategií, a nutnost plánování na krátkodobé i dlouhodobé úrovni**. Teprve kombinace moderních technik strojového učení dokázala tyto výzvy překonat.

Vývoj AlphaStar odráží dlouhodobý směr výzkumu DeepMind. Zakladatel Demis Hassabis již v roce 2011 označil StarCraft za další krok ve výzkumu herní AI po úspěších s AlphaGo, agentem pro hru Go, který porazil světového šampiona. Po spojení s Blizzard Entertainment a zpřístupnění API pro boty v roce 2017 začal DeepMind zpracovávat historická herní data a postupně stavět a trénovat své agentní sítě.

AlphaStar funguje na kombinaci několika technik:

- *Učení z lidských dat*: původní agent byl trénován pomocí imitace strategií z rozsáhlé databáze anonymních her lidských hráčů, aby se naučil základní struktury herního rozhodování.
- *Hluboké zpětnovazební učení (Reinforcement Learning)*: agent dále optimalizoval svou politiku pomocí zpětné vazby ze simulovaných her tak, aby maximalizoval pravděpodobnost vítězství.
- *Multi-agentní trénink a self-play*: agenti hráli proti sobě v rozsáhlých tréninkových ligách, což umožňovalo objevování a adaptaci různých strategií bez ručního navrhování pravidel.
- *Population-Based Training (PBT)*[5]: tato technika automaticky ladila hyperparametry během tréninku, čímž zvyšovala efektivitu a kvalitu učení.

AlphaStar se stal historicky prvním AI systemem, který dosáhl **hodnosti Grandmaster ve veřejném žebříčku hry StarCraft II** bez umělých omezení algoritmu, čímž překonal většinu lidských hráčů v rámci oficiálního kompetitivního prostředí. Tento výkon byl dosažen i přesto, že prostředí kladlo na agenta lidské omezení jako omezené zorné pole kamery a omezenou rychlost akcí, aby se co nejvíce přiblížilo podmínkám, za kterých hrají lidé.

AlphaStar nepředstavuje jedinou strategii, ale udržuje populaci různorodých politik, které odrážejí různé herní styly a strategie. Tato populace se během tréninku dynamicky rozšiřuje a přizpůsobuje, čímž se zvyšuje robustnost a schopnost adaptace agentů v různorodém prostředí. Tento přístup je úzce spojen s principy kvalitní diverzity (Quality Diversity, QD), kdy místo jediné optimalizované strategie vzniká více kvalitních, ale odlišných řešení, která jsou vůči sobě navzájem méně zranitelná.



25

1.2.2 OpenAI Five

OpenAI Five[7] je agent umělé inteligence vyvinutý společností OpenAI, navržený pro hraní týmové videohry Dota 2[4], ve které proti sobě nastupují dvě pětičlenná družstva. Tato hra představuje jedno z nejkompexnějších prostředí pro testování agentů umělé inteligence, protože kombinuje **dlouhé časové horizonty, částečnou pozorovatelnost a vysokodimenzionální akční a stavový prostor**, které výrazně přesahují jednodušší hry jako šachy či Go.

Poprvé se OpenAI Five veřejnosti představuje v roce 2017 během živé demonstrace na turnaji The International, kde v duelu jeden na jednoho poráží profesionálního hráče Dendiho — což ukazuje, že agent dokáže získat přehled a efektivně rozhodovat i v takto náročném prostředí. Následující rok systém pokročuje natolik, že zvládá ovládat celý tým pěti agentů a poráží amatérské i poloprofesionální lidské soupeře ve formátu 5v5.

Vývoj agentních algoritmů pro OpenAI Five začíná již v listopadu 2016. OpenAI zvolila Dota 2 jako testovací prostředí mimo jiné kvůli její **složitosti, nepředvídatelnosti hry, robustnímu API a vysoké popularitě**, což umožnilo testovat a měřit výkon agentů v realistickém konkurenčním scénáři.

OpenAI Five využívá **framework hlubokého zpětnovazebního učení (deep reinforcement learning)**, přičemž jádrem tréninkové metodiky je algoritmus „Proximal Policy Optimization“ (PPO)[8][10]. PPO je gradientní algoritmus politiky, který kombinuje efektivní využití dat, stabilní učení a relativně jednoduchou implementaci — což je důležité při škálování učení na rozsáhlé distribuované systémy, jaké OpenAI v tomto projektu použila. PPO zabráňuje příliš razantním aktualizacím politiky zavedením tzv. oříznutého objektivu (*clipped surrogate objective*), který omezuje změny v distribučních pravděpodobnostech akcí během gradientních kroků.

Každý agent v OpenAI Five je reprezentován neuronovou sítí s jednou vrstvou typu **LSTM o 4096 jednotkách**, která zpracovává aktuální herní stav získaný přes API Dota 2. Tento stav je zakódován jako vektor zhruba 20 000 číselných hodnot, jež popisují relevantní informace z herního prostředí, například pozice jednotek, dostupné akce, cooldowny schopností a další proměnné.

Rozhodování probíhá pomocí několika **nezávislých akčních hlav**, z nichž každá odpovídá za jiný aspekt chování agenta — například volbu konkrétní akce, její umístění v prostoru nebo časování provedení akce. Každý krok agenta je reprezentován osmi číselnými hodnotami (tzv. „action enumeration“), které definují konkrétní volbu v daném časovém okamžiku.

Trénink OpenAI Five probíhá na rozsáhlé distribuované infrastruktuře nazvané Rapid, která umožňuje masivní paralelní výpočet a koordinaci tisíců strojů. Díky tomu systém odehrává denně obrovské množství simulovaných her — v některých fázích tréninku se sbírá ekvivalent více než 180 let herního času na den, což je umožněno paralelním hraním „self-play“ na stovkách GPU a desítkách tisíc CPU jader.

OpenAI Five tak neustále trénuje a optimalizuje své politiky hraním proti sobě samému („self-play“), přičemž model získává odměny za různé klíčové herní cíle, jako je zabíjení nepřátel, ničení struktur nebo dosahování herních objektivů. Tento proces podporuje postupné zlepšování strategie a koordinace celého týmu agentů.

Závěrečnou veřejnou prezentací projektu byly exhibiční zápasy v dubnu 2019, během nichž OpenAI Five poráží tým OG — tehdejší mistry světa z The International 2018 — ve formátu best-of-three se skóre 2:0 na živém streamu, čímž demonstruje, že jeho výkon překonává i špičkové lidské profesionály. Během čtyřdenní online akce OpenAI Five dále otestoval své schopnosti v více než 7 000 zápasech proti veřejným hráčům, z nichž vyhrál přibližně 99,4 % — což potvrzuje konzistentní a robustní výkon v rozsáhlém měřítku.

Díky své robustnosti, efektivitě a schopnosti fungovat v distribuovaných systémech je PPO ideálně vhodný pro tuto aplikaci. Umožňuje agentům provádět rozsáhlý paralelní trénink bez přeučení, přičemž jeho kompatibilita se sdílenými politikami i hodnotovými funkcemi přispívá k úspěšné koordinaci agentů v komplexním prostředí Dota 2.

1.2.3 Voyager

Voyager[13] je zástupce tzv. ztělesněných agentů (embodied agents), kteří kombinují výhody velkých jazykových modelů (LLM) s učením ve složitých, otevřených prostředích. Tento agent funguje autonomně ve hře Minecraft — bez přímého lidského zásahu nebo předem definovaných odměnových funkcí — a učí se postupně z vlastních interakcí s herním světem. Minecraft je pro tento typ agenta ideálním prostředím, protože neobsahuje pevný konec hry, má procedurálně generované světy a vyžaduje dlouhodobé plánování a adaptivní rozhodování, což odráží výzvy reálných úloh.

Klíčové vlastnosti Voyagera jsou založeny na třech hlavních mechanismech, které mu umožňují **otevřené, kontinuální učení bez lidské intervence**:

1. **Automatické kurikulum** – agent si sám generuje sekvenci úkolů na základě aktuálního stavu světa, inventáře a historie již provedených akcí. Tato posloupnost úkolů maximalizuje exploraci a postupně vede ke schopnostem potřebným pro složitější dovednosti.
2. **Knihovna dovedností** – po úspěšném splnění úkolu agent vytvoří program reprezentující tuto dovednost a uloží jej do knihovny. Dovednosti jsou indexovány vektorovými embeddingy jejich popisu, což umožňuje jejich pozdější opětovné použití nebo kombinaci s jinými schopnostmi.
3. **Iterativní promptování** – při řešení úkolu agent nejprve navrhne program (např. v JavaScriptu) pro daný úkol pomocí GPT-4 a poté jej provede ve hře. Výsledek, včetně chyb a zpětné vazby z prostředí, se vrací GPT-4, které program upraví a iterativně opravuje, dokud není úkol úspěšně dokončen nebo vyhodnocen jako neřešitelný.

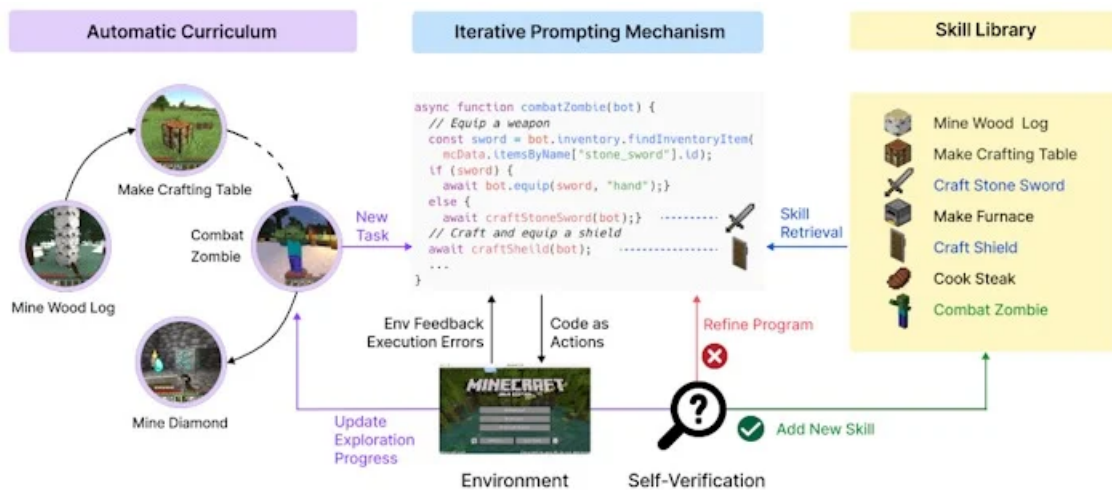
Automatické kurikulum nehledá náhodné úkoly, ale generuje je kontextově na základě aktuální situace v prostředí, což umožňuje efektivní prozkoumávání světa a dlouhodobé zlepšování agentovy schopnosti učit se.

Dovednosti, které Voyager získá, nejsou izolované akce, ale mohou být hierarchicky složeny. Například dovednost vytvořit železný nástroj může využívat dovednosti jako těžba rudy, výroba pece nebo sběr surovin, přičemž se buduje na znalostech získaných dříve a minimalizuje hrazené opakování již naučených postupů.

Iterativní promptování je charakteristické tím, že agent nevykonává pouze jednorázové akce, ale **opakuje cyklus generování a hodnocení kódu** za použití zpětné vazby z prostředí. Kód, který selže kvůli chybě, je upraven na základě analýzy chyby a dalšího promptu, čímž vzniká robustní smyčka učení, která agentovi umožňuje zlepšovat své strategie bez explicitního překalibrování modelu.

Ve srovnávacích experimentech Voyager významně překonává předchozí agentní metody založené na jiných přístupech, jako jsou ReAct či Auto-GPT. Agent získává více unikátních předmětů, pokrývá větší území a rychleji dosahuje technologických milníků, což ukazuje jeho schopnost adaptovat se a učit se v otevřeném prostředí bez ručního ladění cílů.

Celkově lze Voyagera vnímat jako významný krok směrem k obecné autonomní umělé inteligenci v otevřených světech, protože demonstruje, že **agent vybavený LLM a mechanismy samostatného učení může dlouhodobě akumulovat znalosti, generalizovat získané dovednosti a přizpůsobovat se novým úlohám bez lidského zásahu.**



Obrázek 1.6: Vizualizace hlavních komponent systému Voyager – automatického kurikula, knihovny dovedností a iterativního promptování.[27]

Kapitola 2

Použité technologie a návrh hry

2.1 Použité technologie

2.1.1 Unreal Engine

Unreal Engine[18] je multiplatformní herní engine vyvíjený společností Epic Games, který slouží k tvorbě 3D aplikací v reálném čase, včetně videoher, vizualizací, simulací nebo interaktivních prostředí. Jeho jádrem je výkonově orientovaný kód napsaný v jazyce C++, který poskytuje vývojářům přímý přístup k nízkoúrovňovým systémům enginu a vysokou míru kontroly nad chováním runtime prostředí. Unreal Engine je dlouhodobě používán ve vývoji komerčních titulů i ve filmové produkci díky schopnosti zpracovávat složité grafické scény s vysokou vizuální kvalitou.

V projektu je použita verze **Unreal Engine 5.2**, která rozšiřuje možnosti reálného 3D renderování a vývoje prostředí spolu s nástroji pro procedurální generování obsahu a rozsáhlou podporou pro tvorbu světů ve vysokém rozlišení. Verze 5.2 přináší řadu technických vylepšení pro tvorbu reálného času obsahu, včetně nástrojů pro generování velkých scén přímo v enginu a rozšířenou podporu pro materiály a vizuální efekty.

Unreal Engine je navržen tak, aby umožňoval tvorbu komplexních 3D prostředí pomocí kombinace programování v C++ a vizuálního skriptování přes systém Blueprint. Tento přístup umožňuje zachovat výkonově efektivní kód i rychlé prototypování logiky bez psaní rozsáhlých bloků textového kódu. Kromě toho engine poskytuje integrované nástroje pro animace, fyziku, simulace, světelné modely, efekty částic a další subsystémy, které usnadňují implementaci složitých herních či simulačních mechanik.

Unreal Engine nachází uplatnění nejen ve vývoji videoher, ale i v architektonické vizualizaci, filmové produkci nebo průmyslových simulacích, kde je kladen důraz na reálné osvětlení, stíny a materiálové vlastnosti scény. Schopnost kombinovat tyto prvky v reálném čase poskytuje vývojářům platformu pro testování, prototypování i prezentaci výsledků v konzistentním prostředí.

Důvody volby Unreal Engine

Volba prostředí Unreal Engine vychází z jeho předchozího využití v bakalářské práci [32] a z dalších technických důvodů:

1. **Výkon a 3D grafika:** Engine je optimalizovaný pro rozsáhlé 3D scénáře s vysokou vizuální kvalitou a podporuje pokročilé technologie jako Lumen a Nanite, které umožňují dynamické osvětlení a detailní geometrické vykreslení bez výrazných kompromisů na výkonu.
2. **Programování v C++:** Jazyk C++ poskytuje přímý přístup k výkonově kritickým částem enginu a je vhodný pro aplikace, kde je rychlá odezva a efektivní využití hardwaru klíčové; v tomto je výhodou oproti jazykům vyšší úrovně jako C# .
3. **Systém Blueprint:** Vizuální skriptování přes Blueprint usnadňuje tvorbu logiky, interakcí a dalšího chování bez nutnosti hluboké znalosti programovacích jazyků. To snižuje bariéru při vývoji pro části týmu zaměřené více na design než na kódování.
4. **Assety a nástroje:** Unreal Engine integruje nástroje a workflow pro práci s 3D modely, animacemi i zvukem přímo v editoru a podporuje import dat z externích aplikací bez nutnosti ruční konverze.
5. **Rozšiřitelnost:** Engine nabízí systém pluginů a rozhraní API, které umožňuje přidávat vlastní moduly, nástroje nebo rozšíření funkčnosti bez zásahu do hlavní kódové báze projektu.

Nevýhody a technické kompromisy

Použití Unreal Engine oproti jiným enginům přináší i určité technické kompromisy:

1. **Vyšší hardwarové nároky:** Unreal Engine 5 a jeho nástroje pro vysokou grafickou kvalitu obvykle vyžadují výkonnější hardware, zejména při práci s velkými scénami nebo pokročilými efekty. To může být ve srovnání s alternativami jako Unity či Godot náročnější pro vývojové i cílové prostředí.
2. **Složitost a křivka učení:** Vzhledem k rozsahu nástrojů a možností enginu může být jeho zvládnutí náročnější, zejména pro vývojáře bez předchozích zkušeností s C++ nebo komplexními 3D enginy.
3. **Velikost projektu a náročnost správy:** Projekty v Unreal Engine mohou mít větší velikost a složitější strukturu než v jednodušších enginových řešeních, což vyžaduje pečlivé řízení závislostí a assetů během vývoje.



Obrázek 2.1: Ikony použitých technologií

2.1.2 Visual Studio

Visual Studio je integrované vývojové prostředí (IDE) vyvíjené společností Microsoft. Slouží k psaní, kompilaci a ladění programového kódu v různých jazycích, včetně C++.

V kontextu Unreal Engine se Visual Studio běžně používá jako primární nástroj pro vývoj C++ kódu, protože poskytuje přímou integraci s buildovými nástroji, laděním a správou projektů. Podpora Unreal Engine v rámci Visual Studia zahrnuje například možnost rozšířit editor o nástroje pro generování tříd, prohlížení logů nebo debugování kódu vytvořeného z projektu Unreal Engine (pomocí volitelné sady „Visual Studio Tools for Unreal Engine“ a příslušných komponent).

Pro čisté C++ projekty Unreal Engine je Visual Studio doporučený editor, protože usnadňuje generování projektových souborů, správu závislostí a ladění. Unreal Engine poskytuje nástroje pro konfiguraci a synchronizaci s Visual Studií a tento workflow je v oficiální dokumentaci považován za standardní způsob nastavení vývojového prostředí pro C++ programování s Unreal Engine.

V případě tohoto projektu však Visual Studio bohužel nepůsobilo jako použitý nástroj, protože vývoj probíhal bez psaní C++ kódu. Unreal Engine umožňuje tvorbu herní logiky i interakcí prostřednictvím vizuálního systému Blueprint, který nevyžaduje tradiční IDE ani textový C++ editor. V takových případech může být práce nad Unreal projektem realizována přímo v editoru enginu bez zásahu do textového programovacího prostředí. Pokus o úpravu či rozšíření dostupných algoritmů NevarokML nebyl úspěšný z důvodů popsaných v následujících kapitolách této práce.

2.1.3 NevarokML

NevarokML[19] je plugin pro Unreal Engine 5, který umožňuje přímou integraci strojového učení — konkrétně zpětnovazebního učení — do herních nebo simulačních projektů. Plugin je založen na knihovně Stable-Baselines3 a zpřístupňuje oblíbené RL algoritmy jako PPO, A2C, DDPG, DQN, SAC a TD3, které agentům umožňují učit se optimálnímu chování metodou pokus–omyl.

NevarokML podporuje hlavně jednoagentní interakce, kde víceagentní interakce jsou pouze zatím v plánu. Architektura NevarokML funguje na principu klient–server: Unreal Engine zde působí jako serverová část, která simuluje prostředí a chování agentů, zatímco pythonový klient provádí strojové učení pomocí algoritmů ze Stable-Baselines3.

Komunikace mezi Unreal Enginem a Python klientem probíhá prostřednictvím síťového spojení (socketů) s využitím formátu JSON, který slouží k přenosu informací o stavu prostředí, zejména pozorování, akcí a odměn.

Po dokončení trénování jsou výsledné váhy modelu exportovány do formátu ONNX a následně nasazeny přímo do Unreal Engineu jako objekt typu NNModelData prostřednictvím systému Neural Network Engine.

Framework NevarokML tak zajišťuje technickou vrstvu posilovaného učení pro prostředí UE5, zatímco implementace samotného trénovacího modulu a jeho prostředí je plně v kompetenci vývojáře. Trénovací modul musí znát velikost akčního a pozorovacího prostoru a mít přístup k definici prostředí, agenta, kterého řídí, i dalších interagujících objektů.

V samotném prostředí jsou následně definovány konkrétní pozorovací prvky, akční prvky a odměňová logika, které určují, jaká data agent získává, jaké akce může provádět a jakým způsobem je jeho chování hodnoceno.

ONNX[20] (Open Neural Network Exchange) představuje otevřený formát a ekosystém navržený tak, aby umožnil snadný přenos již natrénovaných modelů hlubokého učení mezi různými vývojovými frameworky, platformami a typy hardwaru — bez potřeby složité konverze nebo zdlouhavých procesů.

Neural Network Engine (NNE)[21] je plugin určený pro provádění inferencí neuronových sítí v reálném čase přímo v prostředí Unreal Engine. Plugin využívá formát ONNX, který umožňuje spouštění modelů exportovaných z běžně používaných frameworků strojového učení, jako jsou PyTorch, TensorFlow nebo MXNet, ve formátu .onnx bez nutnosti dalšího převodu.

1. Vytvoření prostředí:

- Práce začíná vytvořením Blueprint třídy děděné z `NevarokMLEnvironment`, kde se definují základní události `OnInit`, `OnStep` a `OnReset`, které řídí logiku prostředí a průběh epizod.

2. Trenérský modul:

- Následuje vytvoření Blueprintu děděného z `NevarokMLTrainer`, kde se nastavují pozorovací a akční prostory (`OnConstruct`) a spouští se samotný trénink (`OnStart`).

3. Implementace v úrovni:

- Trenér i prostředí se umísťují do scény v Unreal Engineu, přičemž trenér odkazuje na prostředí a celý trénink začíná při spuštění úrovně.

4. Sledování učení:

- Pro vizualizaci průběhu učení se může použít `TensorBoard`, který umožňuje sledovat vývoj odměn, změny v politice agenta nebo ladění hyperparametrů.

Plugin je dostupný pro Unreal Engine ve verzi 5.2 a „výše“ (bohužel pro vyšší verze je plugin nepoužitelný kvůli přepisu kódu NNE a taky starému kódu `NevarokML`, který už delší chvíli nebyl aktualizován) a po jeho aktivaci je nutné spustit i backend klienta a ověřit připojení mezi Unreal Enginem a Pythonovým prostředím.

Hlavní vlastnosti a využití:

1. Podpora Blueprintů i C++:

- Plugin lze plně využívat jak přes vizuální skriptování, tak i přes nativní C++ rozhraní pro pokročilejší zásahy (ale není doporučované)

2. Cross-platform nasazení:

- I když trénink typicky probíhá na Windows, natrénované modely je možné nasadit na jakoukoli platformu podporovanou Unreal Enginem.

3. Vhodné pro výuku a výzkum:

- Plugin je volně dostupný pro nekomerční využití, což z něj činí ideální nástroj pro akademické projekty, simulace nebo vývoj prototypů.

2.2 Návrh hry

2.2.1 Návrh prostředí hry

Prostředí, ve kterém bude autonomní agent fungovat, lze realizovat různými způsoby – od jednoduchých po technicky náročnější. Mezi přístupnější varianty patří využití herních enginů, jako je Unity nebo Unreal Engine, zatímco pokročilejší řešení zahrnuje programování vizualizace od základu pomocí nízkoúrovňových API, jako jsou OpenGL nebo DirectX.

V tomto projektu bude vizualizace agenta realizována v prostředí Unreal Engine formou střílečky z pohledu první osoby. Prvním krokem bude vytvoření herní mapy, která umožní volný pohyb hráče i postavy řízené umělou inteligencí.

Mapa bude inspirována městským náměstím po bitvě. Její střed bude tvořit park s bankovní budovou, přičemž park bude vlivem zpusťování téměř prázdný a bankovní budova obklopena barikádami. Právě v této budově se hráč po spuštění hry objeví.

V objektu bude možné nalézt zbraně a zásobovací box obsahující zbraně a brnění, které hráč využije v boji. Bankovní budova spolu s přilehlým parkem obklopeným barikádami bude sloužit jako hlavní bojová aréna pro souboje mezi člověkem a umělou inteligencí. Stejná mapa bude rovněž využita pro souboje mezi dvěma postavami.

Pro vyvážení obtížnosti budou nepřátelé po smrti odhazovat sbíratelné předměty, jako jsou lékárničky a zásobníky munice, které hráč využije k doplnění zdraví nebo střeliva.

Součástí uživatelského rozhraní bude tzv. HUD (Head-Up Display), který bude zobrazovat klíčové herní informace – aktuální stav zdraví, zásoby energie, počet nábojů a další. Z tohoto rozhraní bude možné přistupovat jak k hlavnímu menu, tak k paузovací nabídce.

Celá hra bude strukturována do kol. Po spuštění prvního kola se zavřou dveře banky, přičemž venku bude umístěna další bedna s vybavením, která umožní výměnu zbraní a brnění. V každém kole bude úkolem hráče eliminovat předem daný počet nepřátel. Po jejich porážce se aktuální kolo uzavře a automaticky začne nové – s větším počtem protivníků. Tím vznikne dynamický cyklus, který otestuje schopnosti hráče i chování autonomního agenta v různých podmínkách.

Prostředí a větší množství kódu Blueprintů bylo převzat z mé bakalářské práce[32]. Většina tohoto kódu a i prostředí byla nakonec kvůli potřebným rozsáhlým úpravám a také chybám, co zůstali z dob bakalářské práce, potřeba odstanit (když možné tak upravit), aby bylo prostředí a zbytek kódu použitelný pro nového agenta.

2.2.2 Návrh agenta

Cílem práce je vytvořit agenta řízeného pomocí technik strojového učení, který se dokáže adaptovat v herním prostředí. Agent by měl být schopen se v prostředí zorientovat, přiblížit se ke svému cíli (hráči) a efektivně využít střelnou zbraň k jeho eliminaci.

Agent nebude ovládán předem nadefinovanými skripty, ale bude se učit na základě zkušeností pomocí zpětnovazebního učení. Učící algoritmus je založen na metodě Proximal Policy Optimization (PPO), která je známá svou stabilitou při učení v komplexních a nestacionárních prostředích, jakými jsou moderní videohry. PPO umožňuje efektivní učení politiky i ve vysokodimenzionálním stavovém prostoru a při diskrétním akčním prostoru. Zároveň omezuje velikost aktualizací politiky, čímž zabráňuje nestabilnímu chování během tréninku. Ve srovnání s ostatními algoritmy dostupnými v knihovně NevarokML představuje PPO vyvážený kompromis mezi stabilitou učení, implementační složitostí a praktickou použitelností.

Přibližný algoritmus agenta vytvořeného pomocí NevarokML je schematicky znázorněn na obrázku 2.2. Tento vývojový diagram ilustruje jednotlivé fáze interakce agenta s prostředím — od sběru pozorování, přes předání dat do neuronové sítě, predikci a provedení akce, až po získání odměny, uložení zkušeností a následný trénink modelu.



Obrázek 2.2: Vývojový diagram algoritmu agenta v rámci NevarokML

Přestože framework NevarokML v teorii umožňuje přímou modifikaci architektury neuronové sítě skrze C++, tuto možnost se nepodařilo v rámci práce úspěšně zrealizovat (viz následující kapitola). Veškerá zlepšení jsou proto zaměřena na okolní prvky učící smyčky, zejména na strukturu pozorování, množinu dostupných akcí a definici odměn. Tyto prvky mají přímý vliv na kvalitu získané politiky a zároveň nepřímo ovlivňují i výslednou architekturu neuronové sítě prostřednictvím procesu učení.

Aby měl agent dostatečný přehled o situaci, budou mu prostřednictvím funkce `CollectObservations` předávány relevantní informace, na jejichž základě bude činit rozhodnutí. Tyto pozorování mu umožní vyhodnotit aktuální stav prostředí a adekvátně reagovat.

Agent bude mít přístup ke třem základním typům akcí:

- Pohyb,
- Rotace,
- Střelba – za předpokladu splnění podmínek (např. není aktivní cooldown, hráč se nachází v zorném poli apod.).

Tyto akce budou implementovány ve funkci `ApplyActions`, která interpretuje výstup neuronové sítě jak během tréninku, tak i v nasazení.

Jádrem efektivního učení bude systém odměn implementovaný ve funkci `Handle-Rewards`, jenž určuje, jak bude agent penalizován nebo odměňován za své chování. Bude například odměňován za správnou rotaci, pohyb směrem k cíli nebo udržování optimální vzdálenosti. Zároveň bude trénink omezen tak, aby agent nebyl motivován pouze maximalizovat počet bodů bez reálného splnění cíle. Veškerá motivace bude přímo vázána na výsledek boje.

Na základě tohoto návrhu se očekává, že agent:

1. Rozpozná pozici cíle na základě poskytnutých pozorování,
2. Přiblíží se na efektivní vzdálenost,
3. Srovná rotaci tak, aby směřoval k cíli,
4. Zahájí palbu a pokusí se cíl eliminovat.

Kapitola 3

Implementace hry a agenta, vylepšení jeho algoritmu a porovnání verzí

3.1 Základní logika hry

3.1.1 Hráčská postava

Po spuštění hry se aktivuje úvodní inicializační funkce, která spouští klíčové podsystémy hráčské postavy. Nejprve se připojuje mapa ovládacích vstupů (input map), která umožňuje hráči ovládat pohyb postavy prostřednictvím klávesnice. Následně se k postavě připojuje průhledový displej (HUD), jenž zobrazuje herní informace na obrazovce. Součástí inicializace je také aktivace zaměřovače, který se synchronizuje s HUDem a reaguje na vstupy hráče. Další funkce aktivují obnovovací systémy – konkrétně regeneraci zdraví, energie a kapacity baterie svítilny – a zpřístupňují odpovídající sekce HUDu.

Následující část inicializace vytváří proměnné potřebné pro propojení postavy v blueprintu s logikou sběru předmětů a obsluhy zbraní. Závěrečná fáze této funkce rozšiřuje HUD o prvek, který zobrazuje typ právě vybavené zbraně.

Pohyb kamery hráče se řídí pomocí pohybu myši – horizontálně po (lokální) ose Z a vertikálně po ose Y kamery. Samotný pohyb postavy se řídí dvěma samostatnými směry: vpřed/vzad a do stran. Oba směry využívají směrové vektory, které se následně předávají do funkce zajišťující fyzický posun postavy. Systém současně rozpoznává, zda hráč běží nebo se plíží, aby mohl spustit nebo zastavit odpovídající zvukové efekty.

Pro hladké ozvučení pohybu se využívá makro, které opakovaně volá funkci pro přehrávání zvuků kroků. Pokud hráč přestane padat (například po skoku), automaticky se znovu spouští zvuk chůze. Typ povrchu pod nohama se detekuje pomocí trasovací linky (ray trace) – podle zjištěného materiálu se přehrává odpovídající zvuk kroků.

Běhání hráčské postavy se váže na dostupnou energii (staminu). Pokud hráč není v podřepu ani nestojí na místě, ale začne se pohybovat a disponuje dostatkem staminy, systém automaticky aktivuje běh. Tím se spustí postupné snižování energie a současně se změní tempo přehrávání krokového zvuku. Jakmile běh ustane, energie se začne pozvolna obnovovat a rytmus kroků se upraví odpovídajícím způsobem.

Podobný princip platí pro úskok do stran. Pokud má hráč více než 15 bodů staminy a není v podřepu, může provést úskok ve směru aktuálně stisknuté pohybové klávesy. Po dokončení úskoku následuje krátká pauza, po které se energie začne regenerovat.

Při aktivaci plížení (podřepu) systém ověřuje, že hráč neběží, ani nevykonává úskok, a že má dostatek staminy. Pokud jsou tyto podmínky splněny, postava přechází do podřepu, zpomaluje tempo pohybu a začíná se doplňování energie. Při návratu do vzpřímené pozice se odečte část staminy, ale zároveň se znovu aktivuje její regenerace.

Součástí herní logiky je také systém pozastavení hry. Po stisknutí definovaného tlačítka se hra přeruší a na obrazovce se zobrazí pauzovací nabídka.

Mechanismus spotřeby a obnovy staminy funguje pomocí samostatných funkcí. Během běhu a dalších náročnějších akcí se každým herním tikem odečítá pevně stanovená hodnota staminy. Pokud hodnota klesne na nulu, systém automaticky deaktivuje běh. Pokud hráč zůstává neaktivní, spustí se režim obnovy, který energii doplňuje až do maximální hodnoty.

Hráčská postava disponuje také funkcemi pro správu zdraví. V případě zranění se okamžitě aktivuje odečet životů. Pokud postava stále žije, nastává fáze pomalé regenerace zdraví, která se zastavuje po dosažení maximální hodnoty. Pokud však zdraví klesne na nulu, postava umírá – veškeré ovládání se zablokuje, zvuky se deaktivují a zobrazí se obrazovka smrti s možností ukončení nebo restartu hry.

3.1.2 Zbraňový systém

Zbraňový systém ve hře simuluje chování střelných zbraní realistickým způsobem – včetně střelby, přebíjení, sběru munice a interakce s herním prostředím. Na začátku se inicializuje ukazatel na hráčskou postavu, což umožňuje přímé propojení mezi zbraní a hráčem.

Při výstřelu se automaticky odečte jeden náboj ze zásobníku a zároveň se spustí dvě animace – jedna vizualizuje samotný výstřel, druhá zobrazuje animaci hráče při střelbě. Pokud jsou náboje vyčerpány, zbraň vydává charakteristický zvuk „cvaknutí naprázdno“, který hráče upozorňuje na nutnost přebití.

Přebíjení probíhá pouze tehdy, pokud zásobník není plný. Systém nejprve zjišťuje, kolik nábojů chybí do plné kapacity, a následně je doplní z celkové zásoby. Celý proces doprovází přehrání přebíjecí animace. Když hráč sebere na mapě náboje, systém ověří, zda nebyl překročen maximální limit – pokud ne, munice se přičte.

Interakce se zbraněmi probíhá pomocí dvou specifických tlačítek: klávesa E slouží ke sběru zbraní ze země, klávesa B umožňuje jejich odhození. Při sběru zbraně systém detekuje její přítomnost v okolí hráče, načte informace o typu zbraně, počtu nábojů v zásobníku i zásobě a tyto údaje přenesou do hráčova inventáře. Zbraň se poté odstraní ze scény a nahrazuje se aktivní zbraní, která je vizuálně připojena k hráči.

Při odhození zbraně se aktuálně držená zbraň zničí a před hráčem se vytvoří nová kopie, kterou lze znovu sebrat.

Logika střelby zohledňuje typ právě aktivní zbraně. Pokud má hráč automatickou pušku, hra umožňuje opakovanou střelbu po dobu držení spouště, dokud jsou k dispozici náboje. U brokovnice funguje náhodný rozptyl projektilů, který simuluje široký zásahový úhel typický pro tento typ zbraně – vystřelí se několik střel najednou, každá s mírně odlišnou trajektorií. V případě odstřelovací pušky se aktivuje klasická střelba jedním přesným výstřelem.

Střelbu spouští funkce, která vyhodnocuje rotaci a pozici hráčovy kamery. Ze zbraně se vysílá paprsek (raycast), který simuluje trajektorii střely a může zasáhnout nepřátelskou AI. Celý systém zohledňuje nepřesnosti způsobené pohybem hráče i náhodný rozptyl, zejména u brokovnice.

Po výstřelu se aktivuje simulace zpětného rázu, která upravuje směr pohledu hráče. Pokud střela zasáhne nepřítele, aplikuje se poškození. V případě zásahu objektu s fyzikálními vlastnostmi se na něj přenáší síla v opačném směru vůči směru výstřelu.

Každý výstřel spouští zvukový event, na který mohou nepřátelé reagovat. Zbraň zároveň generuje efekt vyhození prázdné nábojnice – ta vyletí ze zbraně a po určité době mizí ze scény.

Tlačítko R pro přebíjení volá funkci, která spouští proces doplnění zásobníku. Tím se zajišťuje plná integrace mezi hráčovými vstupy a systémem správy munice.

Celý zbraňový systém je úzce propojen s HUDem, který hráči poskytuje vizuální zpětnou vazbu o typu zbraně, aktuálním počtu nábojů a stavu zásob.

Agent využívá obdobný, avšak zjednodušený zbraňový systém – neobsahuje všechny funkcionality a některé modifikační efekty chybí.

3.1.3 Logika řízení herních kol

Logika řízení herních kol a nepřátelských jednotek začíná inicializací klíčových proměnných. Po spuštění úrovně systém nastavuje ukazatel na hráče, obnovuje hlasitost zvukových efektů podle posledního uloženého nastavení a volá funkci pro zahájení nové herní seance.

V rámci této úvodní funkce se resetuje proměnná indikující, zda již byli všichni nepřátelé naspawnováni. Zvyšuje se čítač aktuálního kola a spouští se časovač, který po předem definované době aktivuje začátek daného kola.

Zahájení kola zahrnuje nastavení počtu nepřátel, kteří se mají v daném kole objevit. Následně se spouští časovač, jenž s definovaným zpožděním volá funkci pro výběr náhodného místa spawnování. Na tomto místě se poté vytvářejí instance nepřátelských jednotek.

Současně běží opakovaná kontrolní funkce, která sleduje počet aktivních nepřátel. Jakmile systém detekuje, že jsou všichni eliminováni, aktivuje ukončení aktuálního kola.

Závěrečná funkce každého kola vyhodnocuje vybavení hráče — typ zbraně a nastavené brnění (logika brnění je popsána v další sekci) — a poté spouští další kolo.

Režim AI vs. AI využívá obdobnou řídicí logiku, nicméně v každém kole se vytváří pouze jedna instance AI pro každého vybraného bota a následně se sbírají metriky souboje (např. vítěz a délka trvání). Integrace logiky kol se standardní AI a neuronovou AI byla náročná zejména proto, že všechny tři subsystémy musí sdílet jednotné řízení běhu a synchronizaci stavů, zatímco jejich architektura se výrazně liší.

3.1.4 Systém sbírání předmětů

Logika sběru herních předmětů využívá detekci kolize mezi hráčovou postavou a specifickou kolizní sférou, která obklopuje sbíratelné objekty. Jakmile dojde ke kontaktu těchto dvou komponent, spustí se funkce, která umožňuje interakci a zvednutí daného objektu. Pokud hráč oblast opustí, interakce se přeruší a předmět zůstává nedotčen.

Při sběru zbraní kolizní sféra kromě detekce kontaktu uchovává také informace o konkrétní zbraní – například její typ a počet nábojů v zásobníku i celkové zásobě. Po stisknutí příslušné klávesy sféra tyto údaje předá blueprintu hráče, čímž dojde k přenosu zbraně do jeho rukou a následnému odstranění objektu ze scény.

Pro sběr munice se používá obdobná logika. Jakmile hráč vstoupí do kolizní oblasti s municí, systém ověřuje, zda má dostatek volné kapacity pro daný typ nábojů (automatická puška, brokovnice, odstřelovací puška). Pokud ano, náboje se přičítají do odpovídající zásoby.

V opačném případě zůstávají na místě a mohou být sebrány později, až hráč získá volné místo. Stejný mechanismus se uplatňuje i při sběru lékárníček, které doplní zdraví.

3.1.5 Krabice s vybavením

Pokud hráči dojde munice nebo si přeje změnit typ brnění, stačí přistoupit ke kterékoliv bíle označené krabici a stisknout klávesu E. Otevře se nové herní rozhraní, ve kterém lze pomocí šipek vybrat, zda chce používat brokovnici, automatickou pušku nebo odstřelovačku. Stejně tak lze zvolit typ brnění: lehké, střední nebo těžké.

Střední brnění představuje neutrální variantu. Lehké brnění zvyšuje rychlost pohybu, ale zároveň zvyšuje i zranitelnost. Těžké brnění naopak zvyšuje odolnost, ale snižuje pohyblivost.

Jakmile je hráč s výběrem spokojený, klikne na tlačítko Apply a vrací se zpět do boje.

3.1.6 Herní rozhraní (HUD)

V hlavním menu má hráč k dispozici několik možností. Tlačítko Play spouští samotnou hru. Volba Controls otvírá obrazovku s vizuálně zobrazeným schématem ovládacích kláves. Možnost Options zpřístupňuje nastavení hlasitosti, kvality audia a vizuálních parametrů pro optimalizaci výkonu. Změny lze potvrdit tlačítkem Apply nebo zrušit pomocí Back. Tlačítko Quit Game slouží k ukončení hry.

Během hraní může hráč kdykoli stisknutím klávesy P otevřít pauzovací menu. To obsahuje stejné volby jako hlavní menu, s tím rozdílem, že tlačítko Play zde slouží k návratu do hry a Main Menu přesměrovává zpět do úvodní nabídky. Pokud hráč během hry zemře, zobrazí se speciální nabídka se třemi možnostmi: Restart (nové spuštění hry), Quit to Main Menu (návrat do hlavního menu) a Quit (vypnutí hry).

Hlavní herní HUD je rozdělen do několika oblastí. V levém dolním rohu se nacházejí tři informační lišty: horní zobrazuje stav baterie svítilny na zbrani, prostřední ukazuje úroveň zdraví a spodní indikuje množství energie postavy. Vedle těchto list se zobrazuje číslo aktuálního kola. V pravém dolním rohu je uveden počet nábojů v zásobníku, celkový počet náhradních nábojů a informace o aktuálně držené zbrani (název a typ).

Střed obrazovky je vybaven zaměřovačem, jehož velikost se dynamicky mění v závislosti na rychlosti pohybu hráče, což přispívá k přesnějšímu míření. V horní části displeje se nachází časovač, který odpočítává čas do začátku nového kola. V pravém horním rohu je umístěn panel zobrazující změny atributů nepřátel generované neuronovou sítí, a nad ním se nachází počítadlo snímkové frekvence (FPS) pro sledování výkonnosti hry.

3.1.7 Herní prostředí

Jak bylo uvedeno v předchozí kapitole, herní část mapy se skládá z bankovní budovy, ve které hráč zahajuje hru a připravuje se na nadcházející souboje. Po opuštění budovy se prostřednictvím triggeru automaticky uzavřou dveře a hra tímto okamžikem začíná.

Hráč se následně snaží přežít jednotlivá kola proti botům řízeným umělou inteligencí, jejichž typ je zvolen před spuštěním hry. Souboje probíhají v parku před bankovní budovou, který je obklopen barikádami s cílem omezit herní prostor a zabránit nutnosti zdlouhavého vyhledávání nepřátelských entit.

Stejná mapa je rovněž využita v režimu AI vs. AI, ve kterém hráč vystupuje pouze v roli diváka a sleduje souboj dvou botů řízených vybranými typy umělé inteligence. Zbytek mapy je stylizován jako střed zpustošeného velkoměsta a slouží především k dotvoření atmosféry prostředí.

3.1.8 Nepřátelské postavy

Pro klasickou AI i neuronovou AI byly vytvořeny samostatné herní postavy, jejichž chování je záměrně zjednodušeno, avšak v základních principech odpovídá chování hráčské postavy (např. ztráta životů).

Pro klasickou AI byl implementován jednoduchý strom chování, který napodobuje základní schopnosti neuronové AI, jako je pohyb, střelba či náhodný výběr zbraně/zbroje. Detailní popis tohoto řešení je uveden v následující kapitole.

Postava řízená neuronovou AI obsahuje dodatečné mechanismy, zejména komponentu `NevarokMLCommonSense`, která zprostředkovává komunikaci mezi trénovacím modulem, prostředím a samotnou postavou. Dále jsou využity tzv. *tracery*, tedy neviditelné paprsky vysílané z postavy, jejichž účelem je poskytovat trenéru a prostředí informace o poloze a okolí agenta.

Všechny postavy ve hře, včetně hráče i obou typů botů, jsou vybaveny senzory, aby je klasická AI byla schopna správně detekovat a reagovat na jejich přítomnost.

3.2 Autonomní agent

3.2.1 První verze agenta

Pro začátek testuji samotný proces tréninku, tvorby modelu a následného nasazení natrénovaného modelu do simulace pomocí pluginu NevarokML. V první verzi se rozhoduji vytvořit jednoduchého agenta, který se dokáže pohybovat a v průběhu tréninku i simulace postupně zlepšuje svou schopnost dosáhnout stanoveného cíle.

Trénovací mapa

Na trénovací mapě se nachází hlavní entita typu `NevarokMLTrainer`, která je zodpovědná za inicializaci, koordinaci a správu celého tréninkového procesu v rámci pluginu NevarokML.

Pod touto entitou je 270 podřízených instancí typu `NevarokMLEnvironment`, z nichž každá reprezentuje samostatné simulační prostředí pro jednoho agenta. V každém takovém prostředí dále existuje entita `Agent`, na níž probíhá vlastní trénink nebo simulace podle konfigurace daného prostředí.

Každé prostředí je konstruováno jako otevřená plocha bez ohraničených stěn. Pod podlahou je umístěna spouštěcí oblast typu `Trigger` s názvem `AbyssTrigger`. Tento trigger slouží jako detekční zóna pro případné přetečení agenta mimo hranice validní plochy a může iniciovat reset nebo jinou kontrolní akci v tréninkovém cyklu. V bezprostřední blízkosti každého agenta se nachází druhý trigger s názvem `GoalTrigger`, který představuje cíl simulace — jeho dosažení je vyhodnocováno jako úspěch v daném kroku tréninku.

Při spuštění mapy Unreal Engine nejprve inicializuje všechny herní entity a na pozadí zajišťuje renderování a základní simulaci fyziky. Bezprostředně po načtení mapy však rovněž nastává spuštění tréninkového backendu pluginu NevarokML. Tento backend komunikuje s herní mapou prostřednictvím konzolového rozhraní a naváže spojení mezi NevarokML a běžící instancí Unreal Engine.

Konkrétně backend NevarokML:

- otevře konzoli (příkazovou linku), ve které vypisuje diagnostické informace, stav tréninku, epochy a metriky z aktuálních iterací,
- začne přijímat a odesílat data mezi tréninkovým serverem a jednotlivými instancemi agentů v mapě,
- inicializuje tréninkové smyčky a protokoly komunikace podle nastavení v konfiguračních souborech NevarokML,
- zajišťuje průběžné aktualizace parametrů politiky, hodnotových funkcí nebo jiných komponent učení.

Po navázání komunikace backendu s Unreal Engine začínají agenti automaticky provádět první kroky simulace. V konzoli lze sledovat:

- výstupy o stavu jednotlivých instancí (**episode**, **reward**, **loss**, atd.),
- hlášení o dosažení nebo selhání jednotlivých cílů (například aktivace **GoalTrigger**),
- informace o průběhu tréninku, jako jsou průměrné odměny, změny v politikách nebo hodnotách, nebo případné chyby a výjimky,
- průběžné ukládání checkpointů modelů a jejich verzí.

Z pohledu vizuální simulace se v herním okně Unreal Engine současně s konzolovým výstupem začnou pohybovat agenti ve svých simulovaných prostředích podle jejich aktuálních politik. Agenti vykonávají akce, které jsou výsledkem jejich počátečních parametrů, případně postupně upravovaných během učení. Jejich interakce s prostředím (např. dosažení **GoalTrigger**, proud signálů odměn, aktualizace stavových proměnných) se zpětně promítá do dat, která backend zpracovává a vyhodnocuje.

Nevarok trenér

Entita **NevarokMLTrainer** slouží jako centrální koordinátor tréninku v rámci pluginu **NevarokML**. Jejím úkolem je inicializovat tréninkové prostředí, definovat akční a pozorovací prostory pro agenty a řídit samotný proces učení pomocí zvoleného algoritmu.

Nejdříve se v trenéru nastavuje tzv. *Action Space* (akční prostor). Ten určuje, kolik a jakých typů akcí může agent během jednoho tréninkového kroku provádět. V první verzi, kde agent disponuje pouze základními pohybovými akcemi, akční prostor obsahuje pět definovaných akcí, které jsou prováděny sekvenčně.

Následně se připravuje tzv. *Observation Space* (pozorovací prostor), do kterého agent ukládá relevantní informace o svém okolí a stavu světů simulovaných entitami **NevarokMLEnvironment**. Tyto informace slouží agentovi pro rozhodování a učení během tréninku i následné simulace.

Po inicializaci akčního a pozorovacího prostoru se aktivuje uzel reprezentující algoritmus, který se má použít pro učení. V tomto projektu je pro všechny tréninkové režimy zvolen algoritmus **Proximal Policy Optimization (PPO)**.

Tento uzel definuje klíčové hyperparametry, které ovlivňují chování učení:

- **Policy** = **MlpPolicy** — agent používá plně propojenou vícervrstvou perceptronovou síť pro aproximaci politiky a zároveň (v některých implementacích) hodnotové funkce.
- **Learning Rate** = 0.0003 — řídí velikost kroků při aktualizaci vah modelu; nižší hodnota vede k stabilnějšímu, ale pomalejšímu učení.
- **N Steps** = 200 — počet simulovaných stavů/akcí sbíraných před provedením aktualizace politiky.
- **Batch Size** = 54000 — celkový počet vzorků z pozorovacích dat, které se použijí najednou při tréninkové iteraci.
- **N Epochs** = 200 — počet průchodů tréninkových dat při jedné aktualizaci politiky.
- **Gamma** = 0.9 — diskontní faktor pro budoucí odměny, který ovlivňuje, jak daleko do budoucna agent „vidí“.
- **GAE Lambda** = 0.95 — parametr pro Generalized Advantage Estimation, který vyvažuje bias a variance při odhadu výhodnosti akcí.
- **Clip Range** = 0.2 — rozsah, v němž se změny politiky při aktualizaci omezují, aby se zabránilo nestabilitě učení.
- **Ent Coef** = 0.0 — váha entropické složky ve ztrátové funkci; nulová hodnota nepreferuje vyšší entropii politiky (méně exploração).
- **Vf Coef** = 0.5 — váha hodnotové funkce ve výsledné ztrátě.
- **Max Grad Norm** = 0.5 — parametr pro ořezání (clipping) gradientů, který pomáhá předcházet numerické nestabilitě.

Kromě těchto povinných parametrů uzel PPO nabízí i volitelné nastavení, jako např. **Verbose**, který zapíná detailní výpis tréninkových logů během běhu.

Po definici algoritmického uzlu následuje uzel **Learn**, jenž spouští vlastní trénink a simulaci. Tento uzel obsahuje další klíčové parametry:

- **Algorithm** — zde se připojuje zvolený algoritmus učení, v tomto případě PPO.
- **Device** — volba výpočetního zařízení (např. CPU nebo GPU), na kterém bude model trénován.
- **Timesteps** — celkový počet tréninkových kroků během tréninku.
- **Save Freq** — určuje, jak často se ukládají modely během tréninku.
- **Log Interval** — frekvence zapisování monitorovacích metrik do logů.
- **Save Model Name** — název souboru/modelu, pod kterým se budou trénované modely ukládat.
- **Deterministic** — přístup k deterministickému chování agenta při evaluaci po tréninku.

Dále existují volitelné ladicí volby, které usnadňují vizualizaci a diagnostiku procesu učení:

- **Show Tensorboard** – aktivuje export tréninkových metrik pro zobrazení v TensorBoardu.
- **Show Reward** – zobrazuje průběh odměn během tréninku.
- **Show Step Debug** – umožňuje detailní výpis kroků simulace.
- **Show Reset Debug** – vypisuje informace o provedených resetech prostředí.

V případě, že je tento trenér použit ve hře přímo, je navíc konfigurováno automatické vytváření prostředí a instancí agentů při startu mapy. Trenér poté řídí dané entity v herním režimu stejně, jako by je spravoval během tréninku — tj. zpracovává akce, vyhodnocuje odměny a aktualizuje interní politiky bez dalšího zásahu uživatele.

Všechny uvedené hyperparametry nebyly zvoleny arbitrárně nebo pouze převzaty z literatury, ale byly **laděny experimentálně na základě pozorování chování učení**. Postup byl takový, že se jeden hyperparametr systematicky měnil (zvětšoval či zmenšoval) a po každé změně se vyhodnocoval dopad na průběh tréninku — konkrétně se sledovalo, zda změna vedla k rychlejšímu konvergenci, stabilnějšímu odhadům hodnot, plynulejšímu zvyšování odměn nebo naopak k jejich oscilacím či divergenci. Tento cyklus opakovaných experimentů a vyhodnocování vedl nakonec k sestavení kombinace parametrů, které v testovacích bězích poskytly **nejlepší kompromis mezi stabilitou učení a rychlostí konvergence** v rámci daných tréninkových světů. Takový empirický přístup ke ladění hyperparametrů byl pro tento projekt potřeba, protože kombinace prostředí, akčního prostoru a pozorovacích signálů ovlivňuje samotnou dynamiku tréninkového procesu a nelze ji plně odvodit pouze z teoretických doporučení.

Největší vliv na průběh tréninku má volba samotného algoritmu a politiky. V kontextu tohoto projektu, jak již bylo uvedeno dříve, se ukázal jako nejvhodnější algoritmus **Proximal Policy Optimization (PPO)**. Ze dvou standardních typů politik dostupných v knihovnách, tedy `MlpPolicy` a `CnnPolicy`, jsem **vybral** `MlpPolicy` **z pragmatických důvodů**.

Rozdíl mezi těmito politikami spočívá především v tom, jak zpracovávají vstupy z prostředí. `CnnPolicy` využívá konvoluční neuronovou síť k extrakci rysů z obrazových nebo prostorově strukturovaných dat, což je vhodné především pro vstupy ve formě obrazů nebo vícekanálových matic. Oproti tomu `MlpPolicy` používá plně propojené vrstvy (MLP — multi-layer perceptron), které jsou optimalizované pro zpracování jednoduchých číselných vstupů bez prostorové struktury [?]. V našem případě agent nedostává obrazy ani jiné vícerozměrné struktury, ale pouze numerické hodnoty popisující stav prostředí. Proto použití konvolučních vrstev nepřináší žádnou výhodu a zvyšovalo by složitost modelu bez zlepšení výsledků. Naopak `MlpPolicy` je pro takový typ vstupů přirozenější, vede k rychlejší konvergenci a stabilnější optimalizaci v rámci pozorovaných experimentů.

Naopak nejmenší vliv na konečný výkon modelu mělo nastavení velikosti mini-batchů (tj. kolik vzorků se zpracovává najednou během jedné iterace). U těchto parametrů se projevoval spíše jejich dopad na „rychlost učení“ než na „kvalitu konvergence“ — příliš velké hodnoty vedly sice k mírnému zpomalení tréninku, ale neměly zásadní negativní efekt na dosaženou kvalitu politiky.

Nevarok prostředí

Po inicializaci prostředí se nejprve spouští událost „Event On Init“. Pokud chceme načíst uložený model, aby trénink a simulace nezačínaly od začátku a bylo možné agenta využít kdekoliv, vytvoříme instanci modelu pomocí uzlu „NNEModule“, který model načte a inicializuje. Pokud tento uzel nepoužijeme, vše probíhá od začátku. Uzel však vyžaduje vstupy z „Action Sample“ a „Observation Sample“, aby model věděl, s jakými akčními a pozorovacími daty bude pracovat. Výstup modelu se následně ukládá do proměnné „Model“. V této fázi se také ukládá počáteční pozice agenta do proměnné „Agent Initial Location“, získaná z uzlu „Get Actor Location“. Tato pozice se používá při resetování prostředí na začátku každé epizody. Agent je zároveň přidán do seznamu „Ignore Actors“ pomocí uzlu „Add Unique“, aby byl ignorován při raycast kolizích.

Po inicializaci následuje průběžná simulace skrze událost „Event On Step“. Pokud jsme načetli existující model k použití a dalšímu trénování, musíme jej aktivovat pomocí uzlu „Predict“, který navrhuje akce podle aktuálního stavu prostředí. Tyto akce se aplikují pomocí uzlu „Apply Actions“, zatímco aktuální stav se zaznamenává uzlem „Collect Observations“. Odměny za jednotlivé akce vyhodnocuje funkce „Handle Rewards“, čímž dochází k úpravě modelu.

Mezi hlavními ticky definovanými parametrem „Trainer->TickInterval“ se může spustit událost „Event On Step Skip“. Ta slouží například k aplikaci akcí v jemnějších krocích nebo k vyplnění mezer mezi hlavními kroky. Pozorování získaná během „Step Skip“ se však do tréninkového procesu nezahrnují.

Nakonec se spouští událost „Event On Reset“, která se volá před prvním krokem epizody a po jejím dokončení. Slouží k resetování prostředí do výchozího stavu. Nejprve se aktivuje ladicí uzel „Debug“, který slouží k indikaci poslední získané odměny. Následně uzly „Reset Triggers“ a „Reset Agent“ obnovují stav agentů i spouštěcích oblastí. Uzel „Clear“ vymaže nasbírané akce a pozorování („Action Sample“), aby se nová epizoda mohla začít sbírat od nuly. Na závěr funkce „Collect Observations“ ukládá počáteční stav prostředí, jenž slouží jako výchozí bod pro trénink nebo simulaci.

Tento hlavní řetězec událostí zajišťuje kompletní cyklus tréninku a simulace agenta — od inicializace přes jednotlivé kroky s predikcemi modelu až po resetování prostředí na začátku nové epizody.

Při spuštění sběru pozorování („Collect Observations“) z hlavního cyklu se nejprve nastavuje proměnná „Index Counter“ na výchozí hodnotu (0, 0), která reprezentuje indexy pro iteraci senzorů v mřížce. Tato proměnná se pak postupně inkrementuje pomocí funkčních bloků „Index Plus Plus“, přičemž každé volání posouvá index o jeden krok ve směru osy Y (parametr „Step dx = 0, dy = 1“). Tento mechanismus zajišťuje sekvenční průchod jednotlivými senzory nebo směry detekce.

Během každé iterace se načítá aktuální pozice agenta a cílového senzoru. Pomocí uzlu „Get World Location“ se získává pozice senzoru, a pomocí „Get Actor Location“ se načítá aktuální poloha agenta. Funkce „Find Look at Rotation“ vypočítává rotaci směrem k cílovému bodu, která se následně aplikuje na senzor pomocí uzlu „Set World Rotation“. Tento postup zajišťuje, že senzor vždy směřuje správným směrem před samotným trasováním.

Následně probíhá vzorkování pomocí uzlu „Sample Box Line Trace“, který funguje jako raycast s definovanou délkou a nastaveným kanálem („Trace Channel = Visibility“). Parametry „Difference Multiplier“ a „Velocity Multiplier“ škálují rozdíly vzdáleností a rychlosti, které se ukládají do pozorovacího prostoru. K uzlu jsou rovněž připojeny seznamy „Ignore Actors“ a volitelně „Trace Tags“, jež definují objekty ignorované při kolizi a typy vyhodnocovaných zásahů.

Výstupy z raycastu se zapisují do „Observation Sample“, přičemž obsahují indexy jako „Velocity Length Index“, „Difference Length Index“, „X/Y/Z Velocity Index“, „Direction Index“ a „Normal Index“. Tyto indexy reprezentují složky rychlosti, směrové vektory a normály zásahu, které se následně používají jako vstupní data pro neuronovou síť během tréninku nebo inferencí. Výstupy, které se dále nezpracovávají, směřují na pin „Unused“.

Pro účely vizualizace a ladění slouží volitelné parametry na pravé straně uzlu: „Debug Trace“ umožňuje zobrazení paprsků přímo ve scéně, „Debug Duration“ určuje délku jejich zobrazení, „Debug Color“ nastavuje barvu a „Debug Log“ zajišťuje textový výstup do konzole.

Celý tento mechanismus umožňuje agentovi sbírat strukturovaná pozorování z okolního prostředí prostřednictvím senzorů a line trace. Tato data se následně předávají do pozorovacího prostoru, kde slouží k rozhodování a učení.

Po obdržení pokynu z hlavního řetězce se spouští událost „Apply Actions“, která převádí diskrétní akce agenta na pohyb v herním světě. Nejprve se prostřednictvím uzlu „Enable Movement Tick“ aktivuje pohybový tick komponenty „Nevarok ML Character Input“. Tento krok zajišťuje, že se v aktuálním kroku správně zpracovávají pohybové vstupy.

Následně se pomocí uzlu „Get Discrete Value“ získává hodnota diskrétní akce ze struktury „Action Sample“. Tato hodnota určuje, jakou konkrétní akci agent provede. Výsledná hodnota se následně předává do uzlu „Add Discrete World Move Input“, který ji mapuje na odpovídající směr pohybu v rámci herního prostředí:

- „Left (1)“ — pohyb vlevo,
- „Forward Left (-1)“ — diagonálně vlevo vpřed,
- „Forward (2)“ — vpřed,
- „Forward Right (-1)“ — diagonálně vpravo vpřed,
- „Right (3)“ — vpravo,
- „Backward Right (-1)“ — diagonálně vpravo vzad,

- „Backward (4)“ — vzad,
- „Backward Left (-1)“ — diagonálně vlevo vzad.

V tomto případě je agentovi povolen pouze pohyb vpřed, vzad nebo do stran — diagonální pohyb není možný.

Volba „Grounded Only“ omezuje aplikaci pohybu pouze na situace, kdy se agent nachází v kontaktu se zemí. Po zpracování akce se pomocí uzlu „Force Movement Tick“ vynucuje provedení pohybového snímku, aby se změna polohy okamžitě promítla do simulace.

Tento mechanismus umožňuje převádět diskrétní rozhodnutí neuronové sítě na konkrétní pohyb agenta v prostředí a zajišťuje, že každá akce má přímý vliv v rámci jednoho kroku simulace nebo tréninku.

Po obdržení pokynu z hlavního řetězce se spouští událost „Handle Rewards“, která vyhodnocuje odměny, penalizace a podmínky pro ukončení epizody. Logika této události je rozdělena do několika sekvenčně zpracovávaných částí.

První část představuje časový postih za každý krok („Penalty "Time/Step"“). Pomocí uzlu „Add Reward“ systém agentovi uděluje malou zápornou hodnotu, obvykle $-0,01$, čímž jej motivuje k dosažení cíle co nejrychleji a k minimalizaci zdržení během epizody.

Následuje kontrola podmínek pro ukončení epizody. První z nich představuje dosažení maximálního počtu kroků („Done condition "Time out"“). Systém porovnává počet aktuálních kroků („Steps“) s hodnotou „Max Steps“. Pokud je tato hodnota větší nebo rovna maximálnímu limitu, proměnná „Done“ se nastavuje na hodnotu `true` a epizoda končí kvůli vypršení časového limitu.

Další podmínku představuje pád agenta mimo herní prostor („Done condition "Fell out"“). Detekce probíhá pomocí uzlu „Is Triggered“, který je připojen k „Abyss Trigger“. V případě pádu epizoda okamžitě končí a agent obdrží penalizaci „Penalty "Fell out"“ = $-1,0$, čímž je motivován, aby se vyhýbal situacím vedoucím k pádu.

Poslední vyhodnocovanou podmínkou je dosažení cíle („Done condition "Target reached"“). Uzel „Is Triggered“, připojený k „Goal Trigger“, detekuje vstup agenta do cílové oblasti. Pokud je cíl dosažen, proměnná „Done“ se aktivuje a agent získává kladnou odměnu „Bonus "Target reached"“ = $+1,0$, která podporuje opakování úspěšného chování.

Celá sekvence kombinuje:

- drobnou penalizaci za zbytečné zdržování,
- výraznou penalizaci za neúspěch,
- vysokou odměnu za splnění cíle,

čímž vzniká základní reward shaping, který pomáhá neuronové síti efektivněji učit se správnému chování v herním prostředí.

Součástí logiky prostředí jsou také pomocné funkce, které zajišťují správné resetování a ladění epizod.

Funkce „Reset Triggers“ obnovuje pozici cílového i propastného spouštěče na začátku každé epizody. Nejprve se generuje náhodný vektor v rovině XY pomocí uzlu „XY Random Vector“, který se normalizuje uzlem „Normalize“. Tento směr se násobí škálovacím faktorem (např. 500) a přičítá k aktuální pozici získané přes „Get Actor Location“. Výsledná pozice je nastavena pomocí uzlu „Set Actor Location“. Nakonec se volají uzly „Reset Trigger“ pro oba spouštěče („Goal Trigger“ i „Abyss Trigger“), čímž se zajistí jejich výchozí nastavení pro následující epizodu.

Funkce „Reset Agent“ obnovuje agenta do pozice uložené v proměnné „Agent Initial Location“. Touto pozicí se nastavuje agent pomocí „Set Actor Location“, a následně se uzlem „Reset Input“ vymažou všechna dosavadní přesměrování komponenty „Nevarok ML Character Input“. Tím je zajištěno, že agent vždy začíná každou epizodu z čistého výchozího stavu, bez zbytkového pohybu či vstupů.

Funkce „Debug“ slouží k vizuálnímu i textovému ladění průběhu epizody. Využívá uzel „Draw Debug String“, který zobrazuje nad agentem text s aktuální kumulovanou odměnou („Episode Reward“). Barva textu a délka jeho zobrazení („Duration“) jsou nastavitelné, což umožňuje snadné a rychlé sledování chování agenta během simulace nebo tréninku.

Tyto tři funkce – „Reset Triggers“, „Reset Agent“ a „Debug“ – společně zajišťují spolehlivé resetování prostředí, vynulování stavu agenta a intuitivní vizualizaci průběhu epizody.

Agent, GoalTrigger a AbyssTrigger

Tyto Actors neobsahují žádný vlastní kód — slouží výlučně k zajištění funkčnosti systému NevarokMLEnvironment.

Agent má několik senzorů orientovaných do stran, které sbírají informace o okolním prostředí. Dále obsahuje komponentu NevarokMLCharacterInput, umožňující jeho ovládání z prostředí NevarokMLEnvironment, a dva typy senzorů: standardní NevarokML sensor a sensor pro detekci zásahů.

Actor AbyssTrigger obsahuje obyčejný NevarokML sensor a trigger typu NevarokMLBoxTrigger, který při kontaktu s agentem ukončuje probíhající učící epizodu — jak je popsáno výše. Goal Trigger funguje obdobným způsobem, liší se však svou specifickou funkcí a využitím v herním prostředí.

3.2.2 Vyhodnocení první verze agenta

První verze agenta byla koncipována jako ověřovací model s velmi omezeným chováním. Jejím jediným cílem bylo naučit se základní navigační úlohu, konkrétně přesun k cílové pozici v prostředí bez další interakce. Smyslem této kapitoly je nejen shrnout průběh a výsledky tréninku, ale také analyzovat chování agenta při praktickém testování na herní mapě.

Výsledky tréninku

Během tréninku agent nasbíral celkem 1 080 000 simulovaných kroků a provedl přibližně 3 800 aktualizací modelu při konstantní hodnotě učícího kroku (`learning_rate` = 0,0003). Průměrná délka epizody činila zhruba 6,34 kroků, což odpovídá krátkému a deterministickému úkolu, kde agent rychle dosahoval cílové pozice bez nutnosti složitějšího rozhodování.

Průměrná kumulovaná odměna na epizodu dosahovala hodnoty přibližně 0,937. Tato skutečnost indikuje, že agent dosahoval cíle s vysokou mírou konzistence v rámci definované odměnové funkce. Je však nutné zdůraznit, že tato metrika neodpovídá hernímu úspěchu ve smyslu skutečné hry, ale pouze splnění úzce vymezeného úkolu v tréninkovém prostředí.

Další sledované metriky potvrzují stabilní průběh učení:

`approx_kl` $\approx$ 0,00098 - Velmi nízká aproximovaná KL divergence mezi starou a novou politikou značí, že aktualizace politiky byly malé a plynulé, bez náhlých či nestabilních změn chování.

`clip_fraction` = 0,01, `clip_range` = 0,2 - Drtivá většina aktualizací politiky zůstávala uvnitř klipovací oblasti, což indikuje korektní nastavení algoritmu PPO a stabilní optimalizaci.

`explained_variance` = 0,926 - Hodnotová síť byla schopna vysvětlit přibližně 92 % variability hodnoty stavu, což svědčí o konzistentních a spolehlivých odhadech návratnosti.

`entropy_loss` = -0,157 - Entropická složka zůstávala v rozumném rozmezí, což znamená, že agent si zachovával určitou míru exploração, aniž by docházelo k chaotickému chování.

Hodnoty ztrátových funkcí (`policy_gradient_loss`, `value_loss` i celková `loss`) se pohybovaly v očekávaných mezích a nevykazovaly žádné známky divergence či nestability učení.

Na základě vysokého počtu simulovaných kroků a konzistentních tréninkových metrik lze trénink považovat za konvergentní. Agent se spolehlivě naučil dosahovat cíle definovaného v rámci tréninkové mapy.

Praktické testování na herní mapě

Po dokončení tréninku byl agent nasazen do testovací herní mapy a podroben praktickému hodnocení v herních podmínkách.

Bylo provedeno 50 herních kol proti lidskému hráči. V každém případě se agent pouze přesouval směrem k cílové pozici, bez jakéhokoli projevu taktiky, bojového chování či obrany. Výsledkem bylo, že agent nezvítězil v žádném utkání, tedy dosáhl úspěšnosti 0 %.

Dalších 50 kol bylo odehráno proti standardnímu NPC agentovi, jehož chování je řízeno statickými skripty (behavior tree). Ani v tomto případě agent nezískal žádné vítězství, opět s celkovou úspěšností 0 %.

Tato testování jednoznačně potvrzují, že první verze agenta představuje pouze základní navigační model a nelze ji považovat za plnohodnotnou herní umělou inteligenci. Agent neprojevuje žádné bojové schopnosti, neinteraguje s herními mechanikami, nereaguje na protivníka a není schopen volby vybavení ani jiných strategických rozhodnutí.

Doplňková měření z herních testů

Během praktických testů byly zaznamenány následující hodnoty:

- Průměrná doba do eliminace agenta lidským hráčem: přibližně 4 s.
- Průměrná doba do eliminace agenta standardním NPC: přibližně 10–15 s.
- Způsobení poškození: agent žádné poškození neaplikoval.
- Přesnost střelby a výběr výzbroje: nerelevantní, jelikož agent nedisponuje střeleckým chováním ani volbou zbraní.
- Robustnost vůči latenci a handicapům: chování agenta zůstávalo konzistentní i při umělém zpomalení, omezení zorného pole či jiných handicapech, neboť jeho chování není adaptivní v herním smyslu.

Shrnutí chování první verze

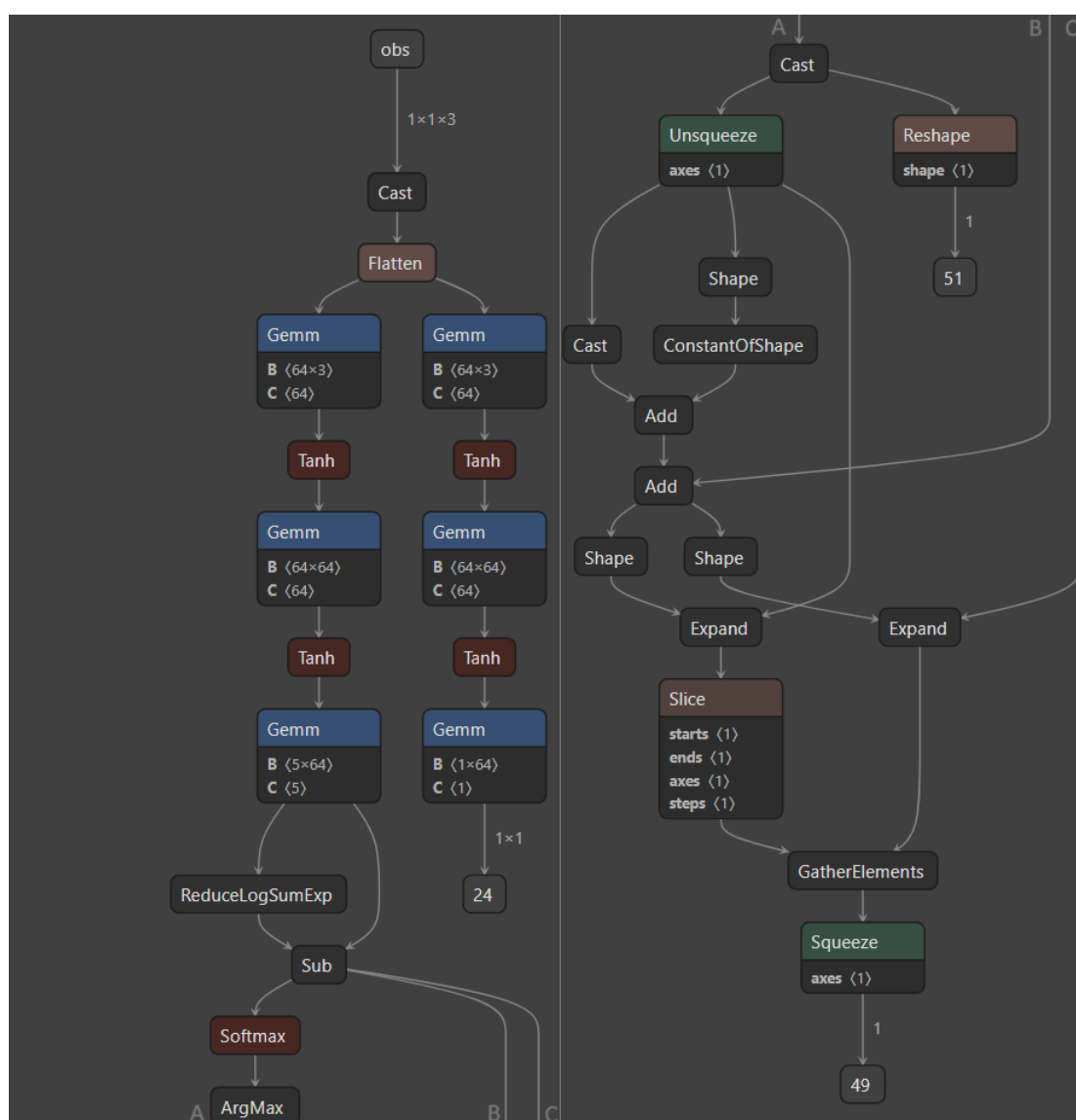
První verze agenta:

- prokázala, že je možné úspěšně natrénovat agenta pro velmi jednoduchý navigační úkol,
- dosahovala stabilních výsledků a vysoké hodnoty odměny v rámci definované odměnové funkce,
- neuměla – a ani neměla umět – herní chování, jako je boj, výběr vybavení či adaptace na protivníka,

- byla výpočetně nenáročná a vykazovala konzistentní chování napříč různými testovacími podmínkami.

Tato verze agenta tedy slouží primárně jako ověření správné implementace tréninkového frameworku, algoritmu PPO, akčního a pozorovacího prostoru. Neřeší však kompetitivní herní situace, které vyžadují bohatší architekturu, širší akční prostor a hlubší reprezentaci stavů. Tyto aspekty jsou adresovány v následujících verzích agenta.

Model a diagram algoritmu první verze agenta



Obrázek 3.1: Exportovaný ONNX model neuronové sítě první verze agenta

Na obrázku 3.1 je znázorněn exportovaný ONNX model neuronové sítě první verze agenta. Jedná se o jednoduchou vícevrstvou perceptronovou síť (MLP) s architekturou typu *actor-critic*, která byla zvolena s cílem ověřit správnost celého tréninkového a inferenčního řetězce.

Vstupní tenzor „obs“ má tvar $1 \times 1 \times 3$ a reprezentuje stavové pozorování agenta. Toto pozorování je nejprve převedeno na odpovídající datový typ pomocí uzlu „Cast“ a následně zploštěno operací „Flatten“ na vektor tří reálných hodnot. Tyto hodnoty odpovídají vzdálenosti agenta od cíle v osách X a Y a maximální odchylce délky, jak je dále schematicky znázorněno na obrázku 3.2.

Po zpracování vstupu se síť větví do dvou samostatných větví odpovídajících politice a hodnotové funkci:

1. Policy head (levá větev):

- Tato část sítě implementuje aproximační model rozhodovací politiky. Obsahuje dvě plně propojené vrstvy realizované operací „Gemm“ s dimenzemi $3 \rightarrow 64$ a $64 \rightarrow 64$, obě s aktivační funkcí „Tanh“. Výstupní vrstva mapuje skrytý stav na pět výstupních neuronů ($64 \rightarrow 5$), které odpovídají pěti diskretním akcím agenta (čtyřsměrný pohyb a nečinnost).

Výstup je dále numericky stabilizován pomocí operace „ReduceLogSumExp“ a normalizován funkcí „Softmax“, čímž vznikne pravděpodobnostní distribuce nad akcemi. Operace „ArgMax“ následně určí akci s nejvyšší pravděpodobností, která je při inferenci provedena v herním prostředí.

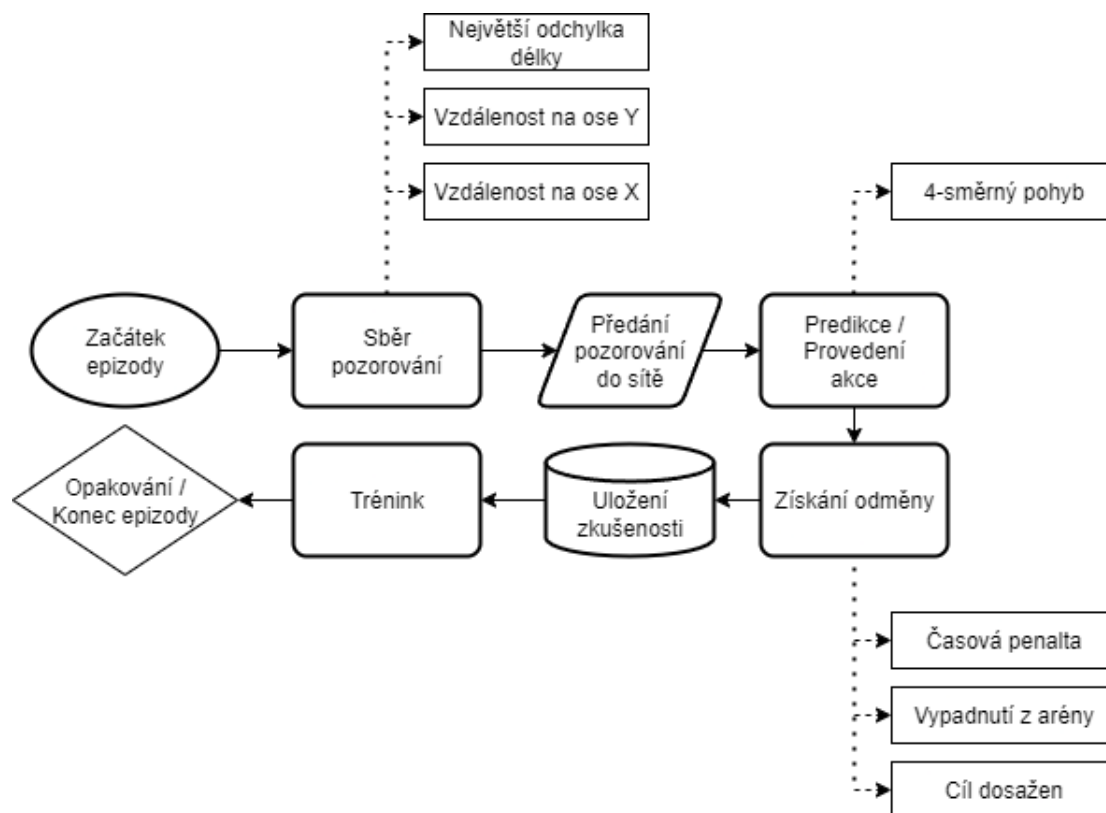
2. Value head (pravá větev):

- Hodnotová větev má identickou strukturu skrytých vrstev ($3 \rightarrow 64 \rightarrow 64$ s „Tanh“), avšak její výstupní vrstva obsahuje pouze jeden neuron ($64 \rightarrow 1$). Tento výstup reprezentuje skalární odhad hodnoty stavu, který slouží jako baseline pro výpočet výhod (*advantage*) v algoritmu PPO.

V pravé části modelu jsou dále patrné pomocné uzly jako „Unsqueeze“, „Shape“, „Slice“ a „GatherElements“. Tyto operace neslouží k vlastnímu rozhodování agenta, ale zajišťují korektní práci s tvary tenzorů a indexování výstupů při paralelním zpracování více instancí prostředí během inferenční fáze.

Architektura modelu přímo odpovídá algoritmickému toku agenta, který je shrnut na obrázku 3.2. Tento diagram ilustruje celý cyklus jedné epizody: od získání stavového pozorování, přes jeho předání do neuronové sítě, predikci a provedení akce, až po získání odměny a uložení zkušenosti pro následný trénink.

Z diagramu je patrné, že chování agenta je v této verzi silně deterministické a omezené výhradně na navigační úlohu. Odměnová funkce zohledňuje pouze časovou penalizaci, dosažení cíle a případné vypaďnutí z arény. Tato zjednodušená logika odpovídá účelu první verze agenta, jejímž cílem nebylo řešit komplexní herní situace, ale vytvořit stabilní a snadno analyzovatelný základ pro další rozšiřování architektury i chování.



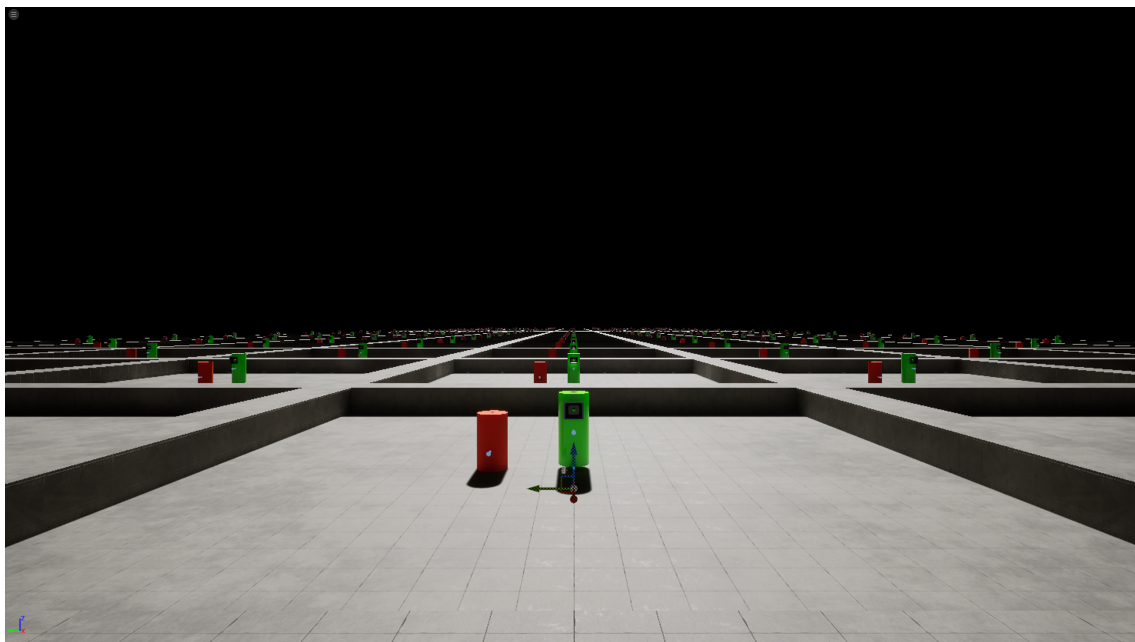
Obrázek 3.2: Vývojový diagram algoritmu agenta – první verze

3.2.3 Druhá verze agenta

Druhá verze agenta již obsahuje kompletní logiku chování – dokáže se pohybovat, otáčet a nakonec i střílí na nepřítele.

Změny v trénovací mapě

Trénovací mapa se částečně mění. Místo podlahy bez stěn má nyní každé prostředí podlahu se stěnami, aby agent během základního tréninku nemůže spadnout mimo vymezenou oblast. Tato oblast je zároveň dvojnásobně větší. Agent na mapě zůstává beze změn, ale obě původní spouštěcí oblasti **GoalTrigger** a **AbyssTrigger** jsou odstraněny. Místo nich se na mapě nově nachází Actor s názvem **Enemy** (nepřítel).



Obrázek 3.3: Vzhled trénovací mapy pro druhou verzi agenta

Změny v trénérovi

V `NevarokMLTrainer` dochází k drobným úpravám. Trénér nyní nenastavuje velikost „Action Space“ pomocí jediné hodnoty, ale přijímá pole o třech prvcích: první s rozsahem 0–8, druhý 0–2 a třetí 0–1. `NevarokML` tak v pozadí vytváří akční krychli, která umožňuje agentovi provádět až tři akce současně namísto sekvenčního provádění. Celkový počet možných kombinací akcí se tím zvyšuje z 5 na 54.

Rozšiřuje se také „Observation Space“ — agent nyní sbírá o dvě další hodnoty navíc, což je nezbytné pro nové typy akcí.

Kromě toho se přidává ukládání počáteční rotace agenta, které se využívá při resetu a pro správnou orientaci v každé nové epizodě.

Změny v prostředí

Ve funkci „CollectObservations“ se přidává logika, která vypočítává vzdálenost a úhlovou odchylku (yaw rozdíl) mezi agentem a nepřítelem. Tyto hodnoty se následně zapisují do pozorovacího prostoru.

Nejprve se pomocí uzlu „Sample Box Line Trace“ detekuje vzdálenost k objektu před agentem. Výstup „Difference Length“ se ukládá do proměnné „Distance To Enemy“ a následně pomocí uzlu „Set Box Value“ do pozorovacího pole.

Současně probíhá výpočet směrového rozdílu mezi agentem a cílem. Uzel „Get Actor Rotation“ spolu s „Break Rotator“ extrahují hodnotu yaw agenta, zatímco funkce „Find Look at Rotation“ určuje požadovaný yaw směr k cíli. Tyto hodnoty se ukládají jako „Agent Yaw“ a „Look at Yaw“.

Úhlový rozdíl se dále normalizuje pomocí uzlu „Normalize Axis“, který zajišťuje, že výsledek zůstává v rozsahu $[-180^\circ, 180^\circ]$. Hodnota odchylky se následně ukládá jako „Yaw Diff“, opět pomocí „Set Box Value“.

Pro každý zápis do pozorovacího pole se využívá inkrementační čítač „Index Counter“, který pomocí funkce „Index Plus Plus“ zajišťuje, že jednotlivé hodnoty se zapisují do různých indexů bez přepisování.

Tato sekvence poskytuje agentovi nové prostorové informace důležité pro navigaci – konkrétně vzdálenost k cíli (nepříteli) a velikost směrové odchylky, kterou musí agent korigovat, aby se správně natočil ke svému cíli.

Funkce „Apply Actions“ nyní zpracovává výstup politiky agenta rozdělený do tří větví podle „Multi Discrete“ reprezentace: pohyb, rotace a střelba.

Pomocí uzlu „Sequence“ se zajišťuje sekvenční provedení všech tří akčních větví:

1. Pohyb:

- První větev zpracovává výstup s indexem 0 pomocí uzlu „Get Multi Discrete Value“ a výsledek předává do uzlu „Add Discrete World Move Input“. Tento uzel interpretuje hodnoty 0–8 jako pohyb vpřed, vzad, do stran a nově také diagonálně.

2. Rotace:

- Druhá větev (index 1) určuje otáčení — hodnoty 1 a 2 reprezentují rotaci vlevo či vpravo, přičemž realizaci zajišťuje uzel „Add Discrete Turn Input“.

3. Střelba:

- Třetí větev (index 2) aktivuje střelbu pouze při splnění dvou podmínek: agent se nachází dostatečně blízko cíli a zároveň je správně natočen. Pokud jsou obě podmínky splněny, vyvolá se uzel „Trigger Fire Action“, který realizuje samotnou logiku střelby.

Každou akci zprostředkovává komponenta „Nevarok ML Character Input“. Nejprve se aktivuje pohybový tick pomocí „Enable Movement Tick“ a po dokončení celé sekvence následuje „Force Movement Tick“, který zajišťuje okamžité aplikování akcí během jednoho simulačního kroku.

Funkce `Trigger Fire Action` představuje zjednodušený systém střelby, který se spouští po uplynutí 0,5s díky uzlu `Do Once` a opakovanému obnovení proměnné `Can Fire` přes `Set Timer by Function Name`. Kód nejprve ověřuje, zda je střelba povolena (`Can Fire == true`); pokud ano, pokračuje dále.

Ze scény se získávají pozice a směr kamery pomocí uzlů `Get World Location` a `Get World Rotation`, přičemž vektor směru se prodlužuje o určitou vzdálenost pomocí operace `+`. Tento výsledný směr se následně použije ve funkci `Line Trace by Channel`, která simuluje paprsek vpřed ve směru pohledu agenta a ignoruje samotného agenta.

Výsledek raycastu se zpracovává pomocí `Break Hit Result`. Pokud dojde ke kolizi (`Blocking Hit == true`) a objekt odpovídá nepříteli (`Hit Actor == Enemy`), agent získává odměnu +2,0 a spouští se funkce `Apply Damage` s hodnotou 10. Pokud je zásah mimo nepřítele nebo k žádnému zásahu nedojde, agent obdrží penalizaci -0,2.

Na závěr se proměnná `Can Fire` nastavuje zpět na `false` a opět se spouští časovač s hodnotou 0,5s. Po jeho vypršení se střelba opět povoluje.

V novém systému odměn je agent průběžně motivován k dosažení cíle, správné orientaci, udržování optimální vzdálenosti od cíle a eliminaci nepřítele, přičemž je penalizován za chyby, neaktivitu nebo vypršení časového limitu.

1. Odměna za správné natočení:

- Pokud je rozdíl směru („YawDiff“) mezi agentem a směrem k nepříteli malý, agent získává odměnu až do výše 0,03 — čím přesněji míří, tím vyšší odměna. Při větším odchýlení ztrácí až -0,05.

2. Odměna za rotaci během pohybu:

- Agent je odměňován pouze tehdy, pokud se pohybuje vpřed a zároveň se otáčí směrem k cíli, což se detekuje pomocí skalárního součinu forward vectoru a směru k cíli. Odměna se škáluje až do výše 0,03.

3. Odměna za ideální vzdálenost:

- Agent získává mírnou odměnu (maximálně 0,03) za udržování optimální vzdálenosti od cíle. Pokud je příliš blízko nebo příliš daleko, je penalizován hodnotou -0,03.

4. Eliminace nepřítele:

- Pokud agent úspěšně eliminuje nepřítele, epizoda se ukončuje a je mu přidělena hlavní odměna ve výši 5,0.

5. Penalizace za timeout:

- Pokud agent epizodu nevyřeší a dosáhne maximálního počtu kroků, obdrží výraznou penalizaci -10,0, která slouží jako jasný signál selhání.

Všechny odměny v této verzi i v předchozích či nadcházejících jsem nevolil arbitrárně, ale **experimentálně dle pozorovaného chování agenta**. Při ladění odměnové funkce jsem sledoval, jaké chování agent vykazuje a jak se jeho strategie v průběhu učení mění. Když agent vykazoval nechtěné chování — například opakovaně preferoval neefektivní pohyb, stagnoval, nebo využíval nežádoucí exploity prostředí — upravoval jsem **konkrétní složky odměny** tak, aby bylo toto chování potlačeno a aby agent více preferoval žádoucí vzorce chování.

Tento přístup vyžaduje postupné testování a hodnocení. Pro každý návrh odměňové funkce jsem spustil sérii tréninků a sledoval průběh odměn, stabilitu učení, změny politiky a výsledné trajektorie akcí. Pokud některá úprava vedla k nižší stabilitě nebo k přetrvávání nežádoucích vzorců, byla dále upravena.

V konkrétním případě popisované verze se ukázalo, že původní odměňování samotné obsahovalo chybu nebo bylo nastaveno nevhodně pro cíle dané simulace. Tato chyba se projevovala tak, že agent nevykazoval očekávané učení ani po delší době tréninku — nikoli proto, že by samotný algoritmus PPO selhával, ale proto, že **odměňová funkce neodrážela správně žádoucí chování**. Po identifikaci tohoto problému bylo nezbytné odměňovou strukturu upravit tak, aby odpovídala cílovým metrikám učení a vedla agenta ke stabilní konvergenci k žádoucí politice.

Funkce „Reset Triggers“ je nahrazena funkcí „Reset Enemy“, která funguje obdobně, avšak navíc nastavuje poškozenému nepříteli proměnnou „Death“ na `false` a obnovuje jeho zdraví na hodnotu 50.

Funkce „Reset Agent“ se kromě korekce počáteční rotace agenta nemění.

Agent samotný zůstává beze změn, nicméně je vytvořen nový actor „Enemy“, jenž obsahuje základní „NevarokML“ senzor a také kód pro zpracování poškození a případného úmrtí. Při zásahu se spouští událost `Event AnyDamage`, která volá vlastní událost `Health Modifier` s předanou hodnotou poškození. Tato událost nejprve odečítá hodnotu `Damage` od aktuální proměnné `Health` a poté výsledek ořezává pomocí funkce `Clamp (Float)` na rozsah `[0,0;Max Health]`, aby nedošlo k přetečení nebo záporné hodnotě. Upravená hodnota se ukládá zpět do proměnné `Health`.

Následuje kontrola, zda je zdraví menší nebo rovno nule. Pokud ano, aktivuje se větev `True`, která spouští uzel `Do Once`, a následně vlastní událost `Dead`. Tím se proměnná `Death` nastavuje na `true`, což signalizuje zabití nepřítele. Při vyhodnocení odměn to automaticky spouští reset prostředí.

3.2.4 Vyhodnocení druhé verze agenta

Druhá verze agenta představuje první variantu, která již neřeší pouze navigaci k cíli, ale zahrnuje i základní bojovou interakci. Oproti první verzi tedy agent musí protivníka lokalizovat, zaujmout vůči němu vhodnou orientaci, zahájit útok a v ideálním případě jej eliminovat. Tomu odpovídá jak delší průběh epizod, tak i vyšší složitost optimalizace.

Výsledky tréninku

Agent během tréninku dosáhl celkem 3 240 000 kroků a model byl aktualizován $11\,600\times$ při konstantním učícím kroku (`learning_rate` = 0,0003). **Nejlepší pozorované chování** se však v praxi projevilo přibližně okolo 2,5 milionu kroků, tedy před dokončením celého tréninkového běhu; v pozdější fázi se již zlepšení neprojevovalo konzistentně, či byl dokonce agent horší. Tento bod byl určen empiricky na základě praktických herních testů (stabilita chování a výsledků), nikoliv pouze podle hodnot tréninkových metrik. Průměrná délka epizody vzrostla na `ep_len_mean` = 36,4, což je očekávatelný důsledek rozšíření úlohy o vyhledání cíle, bojovou akci a vyhodnocení výsledku střetu.

Průměrná kumulovaná odměna na epizodu dosahovala `ep_rew_mean` = 13,8. Na rozdíl od první verze tato odměna již odráží kombinaci více signálů (např. detekce cíle, zásahy, penalizace za nevhodné chování a časová složka), a proto je její interpretace nutně více závislá na konkrétní konstrukci odměnové funkce.

Z hlediska stability a dynamiky učení jsou podstatné následující metriky:

`approx_kl` = 0,00734 - Přibližná KL divergence je výrazně vyšší než u první verze, což odpovídá agresivnějším aktualizacím politiky při učení komplexnějšího chování. Zároveň se stále pohybuje v rozumném rozsahu pro PPO, tj. nedochází k nekontrolovaným skokům v politice.

`clip_fraction` = 0,0368, `clip_range` = 0,2 - Vyšší podíl klipovaných aktualizací je přirozeným důsledkem silnějších gradientových signálů. PPO zde plní svou roli stabilizátoru tím, že tlumí příliš velké kroky v parametrech politiky.

`entropy_loss` = -0,968 - Výrazně nižší entropie indikuje pokles náhodnosti a přechod k determinističtějšímu rozhodování. V kontextu bojové úlohy to typicky znamená, že agent konverguje k relativně konzistentní strategii.

`explained_variance` = 0,499 - Nižší vysvětlená variabilita ukazuje, že hodnotová síť (kritik) predikuje návratnost méně přesně. Tento jev je očekávatelný u úloh s více zdroji odměn a vyšší stochastičností; zároveň však zvyšuje citlivost algoritmu na kvalitu návrhu odměn.

Hodnoty ztrát dále dokládají, že optimalizace je oproti první verzi náročnější:

`policy_gradient_loss` = -0,00472 naznačuje silnější tlak na změny politiky, zatímco `value_loss` = 1,02 odpovídá obtížnějšímu učení hodnotové funkce. Celková `loss` = 0,501 přitom neindikuje divergenci, ale spíše typické kompromisy při souběžném tréninku aktéra a kritika v komplexnějším prostředí.

Z praktického hlediska se v této verzi projevuje limitace návrhu odměn (zejména ve funkci „Handle Rewards“). Agent má tendenci ignorovat preferovanou bojovou vzdálenost a přibližovat se k protivníkovi „co nejbližší“, pokud to krátkodobě maximalizuje pravděpodobnost zásahu. Příčina je přímočará: odměna za zásah v aktuálním nastavení převyšuje penalizaci za nevhodnou vzdálenost, takže optimum politiky vzniká ve směru „agresivního dotlačení“ na cíl. Z hlediska herního kontextu jde o nežádoucí chování, které se následně promítá do výsledků proti některým typům soupeřů.

Praktické testování na herní mapě

Po dokončení tréninku byl agent nasazen do testovací herní mapy a hodnocen ve třech typech střetů: proti lidskému hráči, proti standardnímu NPC agentovi (behavior tree) a proti první verzi neuronového agenta (NAI V1).

Testy proti lidskému hráči (50 pokusů). Namísto klasického vyhodnocení „50 herních kol“ bylo v praxi provedeno 50 samostatných pokusů, protože agent v této verzi dokázal hráče eliminovat opakovaně (tj. ve více po sobě jdoucích kolech) v rámci jednoho běhu. Typicky se ukázalo, že k eliminaci hráče docházelo po 5–6 kolech, což odpovídá situaci, kdy proti hráči postupně stojí 5–6 instancí agenta. Průměrná doba eliminace jedné instance agentem–hráčem byla přibližně 5 s. Agent zároveň dokázal způsobovat poškození; průměrné udělené poškození vůči hráči bylo přibližně 21.

Testy proti standardnímu NPC (50 kol). Proti klasickému NPC dosáhl agent přibližně 45 % úspěšnosti. Nižší úspěšnost souvisela především s výše popsaným chováním: v některých epizodách agent ignoroval bezpečnou vzdálenost, přiblížil se k protivníkovi na minimální odstup a stal se snadným cílem. Průměrná délka kola se pohybovala kolem 10 s a průměrné udělené poškození činilo přibližně 50 ze 100.

Testy proti NAI V1 (50 kol). Ve střetech proti první verzi neuronového agenta dosáhla druhá verze 100 % úspěšnosti (50/50). Tento výsledek je konzistentní s rozdílem schopností: zatímco NAI V1 disponuje pouze navigací, NAI V2 již aktivně vyhledává cíl, natáčí se a útočí. Průměrná délka kola byla přibližně 10 s a průměrné udělené poškození dosahovalo 100, tj. plné eliminace protivníka.

Doplňková měření z herních testů

Během praktických testů byly pozorovány následující vlastnosti a limity:

- **Přesnost střelby:** průměrná přesnost se pohybovala přibližně v rozmezí 50–65 %. Záznamy chování ukazují, že část nepřesnosti souvisí s problémem rotace: agent se při zaměřování místy „cukal“ a korekce směru nebyly plynulé.
- **Výběr výzbroje a brnění:** agent v této verzi stále nedisponuje mechanikami pro změnu zbraně ani brnění, a proto tyto aspekty nelze hodnotit.
- **Latence a handicap:** oproti první verzi se projevila degradace výkonu při uměle zavedené latenci (zpoždění), což je konzistentní s větší závislostí na časově přesné orientaci a střelbě. Naopak omezení zorného pole (FOV handicap) v této verzi nevedlo k jednoznačně pozorovatelnému zhoršení.

Shrnutí chování druhé verze

Druhá verze agenta:

- rozšířila schopnosti z čisté navigace na základní bojovou interakci (vyhledání cíle, orientace, útok),
- dosáhla stabilního tréninku, avšak s očekávatelně vyšší náročností učení hodnotové funkce,
- prokázala schopnost aktivně způsobovat poškození a konzistentně porážet NAI V1, ale ne porazit klasické AI,
- vykazovala neoptimální chování vyplývající z návrhu odměn (příliš agresivní přiblížení), což se negativně projevilo zejména proti standardnímu NPC,
- zůstává omezená absencí výběru zbraní a brnění a citlivostí na latenci.

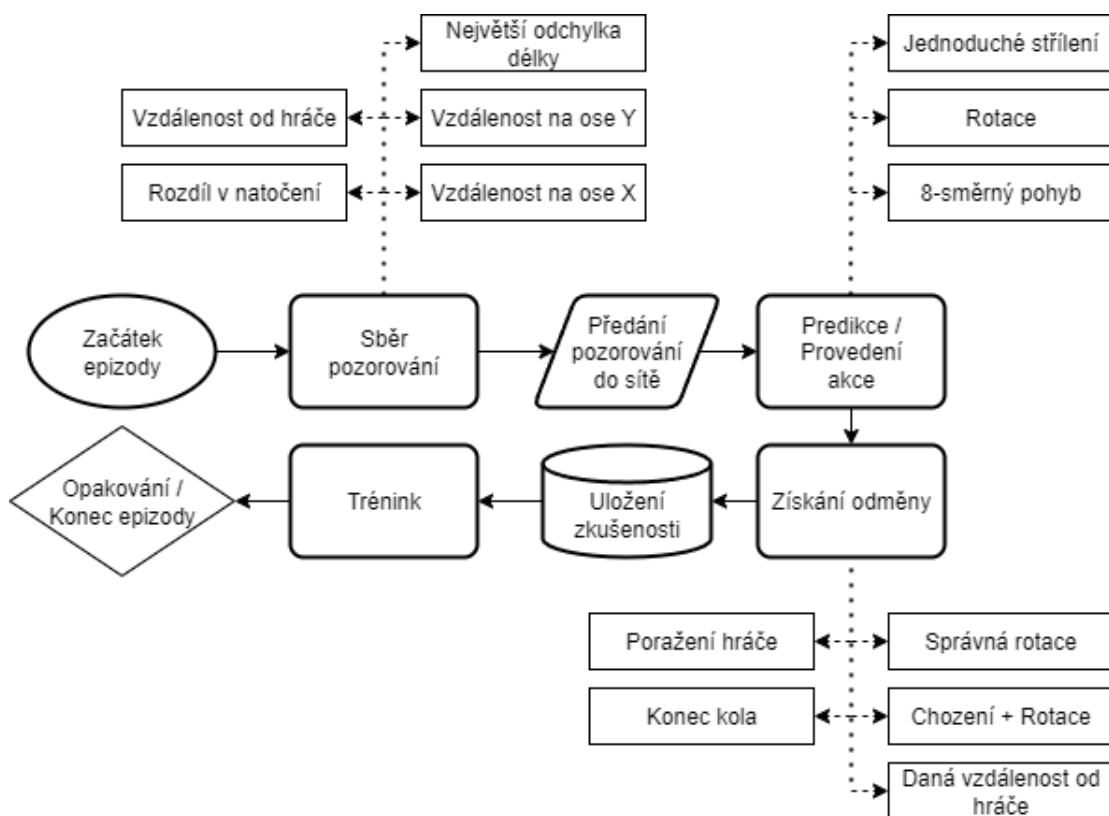
Diagram algoritmu druhé verze agenta

Na rozdíl od první verze je z vývojového diagramu na obrázku 3.4 patrné výrazné rozšíření pozorovacího i akčního prostoru agenta. Fáze „Sběr pozorování“ již nezahrnuje pouze základní navigační informace, ale také veličiny přímo související s bojovou interakcí. Konkrétně se jedná o vzdálenost od hráče, rozdíl v natočení (relativní orientaci) a složky vzdálenosti v osách X a Y , doplněné o maximální odchylku délky.

Z diagramu je zároveň patrné, že výstupní akce již nejsou omezeny pouze na pohyb, ale zahrnují osmisměrný pohyb, rotaci a jednoduché střelecké chování. Tím se agent poprvé dostává do situace, kdy musí koordinovat více typů akcí současně.

Odměnová část diagramu explicitně rozlišuje několik dílčích signálů, zejména správnou rotaci vůči hráči, udržení určité vzdálenosti a samotné zásahy. Právě tato struktura vysvětluje některé pozorované projevy chování: agent má tendenci optimalizovat krátkodobý zisk z úspěšného zásahu i za cenu porušení preferované vzdálenosti, pokud je váha odměny za zásah dominantní. Diagram tak názorně ukazuje, že nežádoucí chování není důsledkem chybné implementace neuronové sítě, ale přímo vyplývá z konstrukce odměnové funkce.

Na rozdíl od první verze je zde rovněž zřetelnější vazba mezi jednotlivými kroky epizody. Akce agenta přímo ovlivňují další pozorování, která jsou ukládána do paměti zkušeností a následně využívána při tréninku. Tento uzavřený zpětnovazební cyklus je u druhé verze výrazně dynamičtější, což se promítá jak do delší délky epizod, tak do vyšší citlivosti na zpoždění (latenci) při inferenci.



Obrázek 3.4: Vývojový diagram algoritmu agenta – druhá verze

3.2.5 Finální verze agenta

Modifikace prostředí

Jednoduchého agenta ze základního tréninku nahrazuje agent označovaný jako NPC, který využívá propracovanější střeleckou logiku přímo implementovanou na zbraních navržených speciálně pro něj. Odměna za zásah je snížena, což zabraňuje agentovi získávat body tím, že se přiblíží k hráči na extrémně krátkou vzdálenost a opakovaně jej zasahuje. NPC agent má schopnost volby mezi třemi typy zbraní, čímž získává vyšší funkční flexibilitu.

Dále dokáže přijímat poškození — pokud mu klesne život na nulu, umírá a na jeho místě se objeví lékárnička a munice, jak bylo popsáno výše. Před každým novým kolem si agent zapamatuje zbraň a brnění, které hráč použil v předchozím kole, a na základě těchto informací volí optimální kombinaci výbavy pro kolo následující.

V odměnové logice je přidáno nové omezení a je upraveno jedno existující:

1. Kombinovaná odměna:

- Pokud jsou současně splněny podmínky správného natočení a optimální vzdálenosti od cíle, agent získá odměnu $+0,1$. V opačném případě obdrží penalizaci $-0,1$.

2. Eliminace nepřítele:

- Pokud agent úspěšně eliminujete nepřítele, epizoda končí a připisuje se hlavní odměna. Výchozí hodnota činí $5,0$, ale násobí se koeficientem závislým na aktuální vzdálenosti v okamžiku zásahu – nejvyšší odměna je udělena při optimálním odstupu od cíle.

Nakonec byla upravena i entita actor „Enemy“ tak, aby se aktivně pohybovala po trénovací mapě a postupně — i když s nižší rychlostí — střílela na tréninkového agenta. Tento režim je navržen tak, aby bylo prostředí náročnější, a agent tak nečelil pouze statickému protivníkovi, který se spawnuje na místě a pasivně čeká, zda jej agent zasáhne.

3.2.6 Vyhodnocení finální verze agenta

Finální verze agenta představuje nejkomplexnější variantu implementovanou v rámci této práce. Oproti druhé verzi již agent nejen střílí a rotuje, ale zároveň stabilněji udržuje vhodnou vzdálenost od protivníka a je schopen využívat jednoduchou adaptaci výbavy mezi koly. To se projevuje jak ve stabilnějších tréninkových metrikách, tak ve zlepšených výsledcích praktických herních testů.

Výsledky tréninku

Trénink proběhl po dobu 3 240 000 kroků, při 11 600 aktualizacích modelu a konstantní hodnotě učicího kroku (`learning_rate` = 0,0003), kde poslední krok byl také nejlepší. Průměrná délka epizody byla `ep_len_mean` = 31,4, což je mírně méně než u druhé verze (36,4). Tento pokles odpovídá efektivnějšímu průběhu epizod: agent rychleji navazuje kontakt s protivníkem, stabilněji se orientuje a častěji dokončuje střet bez zbytečných prodlev.

Průměrná kumulovaná odměna na epizodu dosahovala `ep_rew_mean` = 9,2. Nižší hodnota oproti druhé verzi (13,8) je v tomto případě očekávatelná a konzistentní s přísnějšími podmínkami odměňování, které více penalizují nevhodné chování a omezují exploitační strategie.

Z hlediska stability učení jsou klíčové následující metriky:

`approx_kl` = 0,00414 Přibližná KL divergence je nižší než u druhé verze (0,00734), což značí méně agresivní a stabilnější aktualizace politiky.

`clip_fraction` = 0,0332, `clip_range` = 0,2 Podíl klipovaných aktualizací zůstává v podobném rozsahu jako u druhé verze, což odpovídá tomu, že PPO stále aktivně omezuje příliš velké kroky, avšak bez známek nestability.

`entropy_loss` = -0,994 Další pokles entropie oproti druhé verzi indikuje vyšší determinističnost rozhodování, což se v praxi projevuje menším jitterem a stabilnějším zaměřováním.

`explained_variance` = 0,604 Hodnota je vyšší než u druhé verze (0,499), což ukazuje na zlepšení predikce návratnosti hodnotovou sítí.

`value_loss` = 0,7 Pokles oproti druhé verzi (1,02) naznačuje menší nestabilitu kritika a konzistentnější odhady hodnoty stavu.

Celkově lze finální trénink hodnotit jako stabilnější než u druhé verze: politika se aktualizuje méně agresivně, hodnotová síť predikuje návratnost spolehlivěji a chování v praxi působí konzistentněji.

Praktické testování na herní mapě

Finální verze agenta byla testována ve třech typech střetů: proti lidskému hráči, proti standardnímu NPC a proti druhé verzi neuronového agenta (NAI V2).

Testy proti lidskému hráči (50 pokusů). V rámci 50 samostatných pokusů docházelo typicky k eliminaci hráče ve 4. kole. Oproti druhé verzi agent působil výrazně přesněji a stabilněji: rotace byla plynulejší (bez výrazného „cukání“) a agent si častěji udržoval vhodnou vzdálenost od cíle, což snižovalo možnost hráče využít krátkého odstupu k rychlé eliminaci. Průměrné poškození způsobené jednou instancí agenta bylo přibližně 40.

Testy proti standardnímu NPC (50 kol). Proti klasickému NPC agent dosáhl přibližně 55 % úspěšnosti. Průměrná délka kola byla okolo 10 s a průměrné udělené poškození činilo přibližně 50–55. Ve srovnání s druhou verzí je patrné, že agent již častěji volí efektivnější chování (zejména v práci se vzdáleností) a méně často se dostává do situací, kdy je snadno eliminovatelný.

Testy proti NAI V2 (50 kol). Ve střetech proti druhé verzi neuronového agenta dosáhla finální verze úspěšnosti přibližně 55–60 %. Délka kola se pohybovala kolem 10–12 s a průměrné udělené poškození činilo přibližně 55–60. Výsledek odpovídá tomu, že finální verze je vůči V2 stabilnější a přesnější, avšak stále nejde o absolutně dominantní strategii.

Doplňková měření z herních testů

- **Přesnost střelby:** průměrná přesnost se pohybovala přibližně v rozmezí 75–80 %. Přetrvávají pouze mírné problémy v oblasti rotace, avšak výrazně menší než u druhé verze.
- **Adaptace výbavy:** agent dokázal v dalším kole zvolit výhodnou kombinaci zbraně a brnění přibližně v 80–90 % případů. Chybná volba se objevovala jen sporadicky a typicky nepůsobila jako konzistentní systematická chyba.
- **Latence a FOV:** zavedená latence výkon mírně zhoršovala, avšak v menší míře než u druhé verze. Změna zorného pole (FOV) nevedla k pozorovatelnému zhoršení chování.

Shrnutí chování finální verze

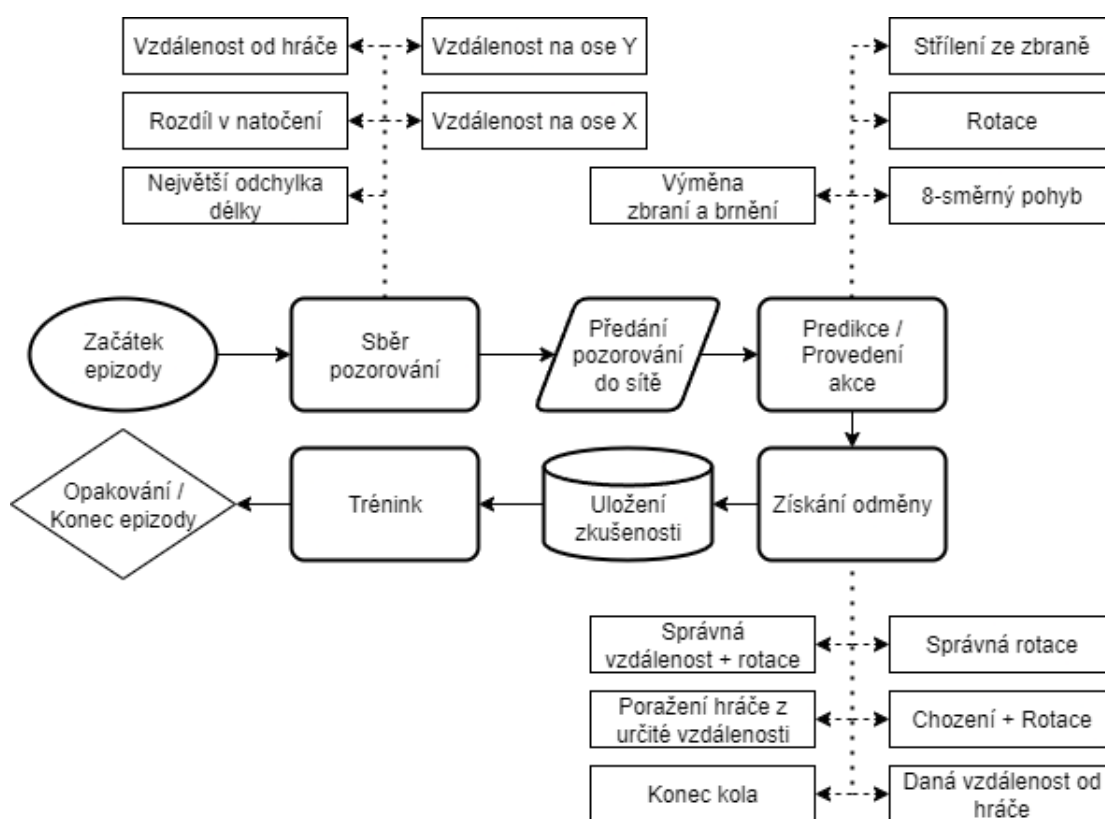
Finální verze agenta:

- vykazovala stabilnější trénink než druhá verze
(nižší `approx_kl`, vyšší `explained_variance`, nižší `value_loss`),
- v praxi působila přesněji a konzistentněji (vyšší přesnost střelby a menší jitter v rotaci),
- dokázala adaptovat výbavu s vysokou úspěšností (80–90 % správných voleb),

- dosahovala lepších výsledků proti standardnímu NPC i proti NAI V2 (cca 55–60 % úspěšnosti),
- zůstává mírně citlivá na latenci, avšak méně než druhá verze, přičemž FOV změny chování neovlivnily.

Diagram algoritmu finální verze

Obrázek 3.5 zachycuje vývojový diagram finální verze. Oproti druhé verzi je patrné doplnění akce pro výměnu zbraní a brnění a rozšíření odměnové logiky o kombinované podmínky (správná vzdálenost a rotace). Diagram tím přímo odráží dvě prakticky důležité vlastnosti finální verze: stabilnější udržování odstupu a schopnost adaptovat výbavu podle předchozího kola.



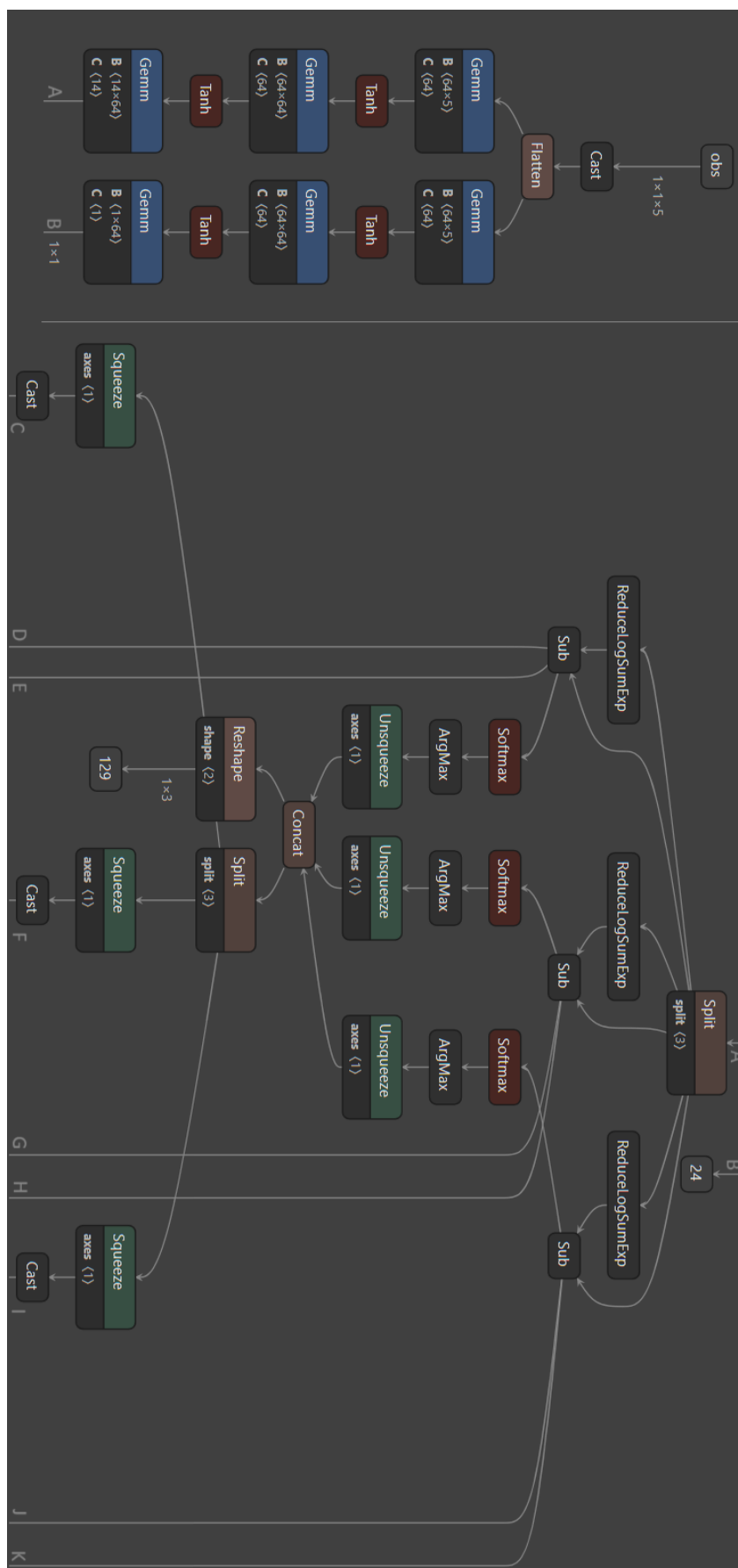
Obrázek 3.5: Vývojový diagram algoritmu agenta – finální verze

Model finální verze agenta

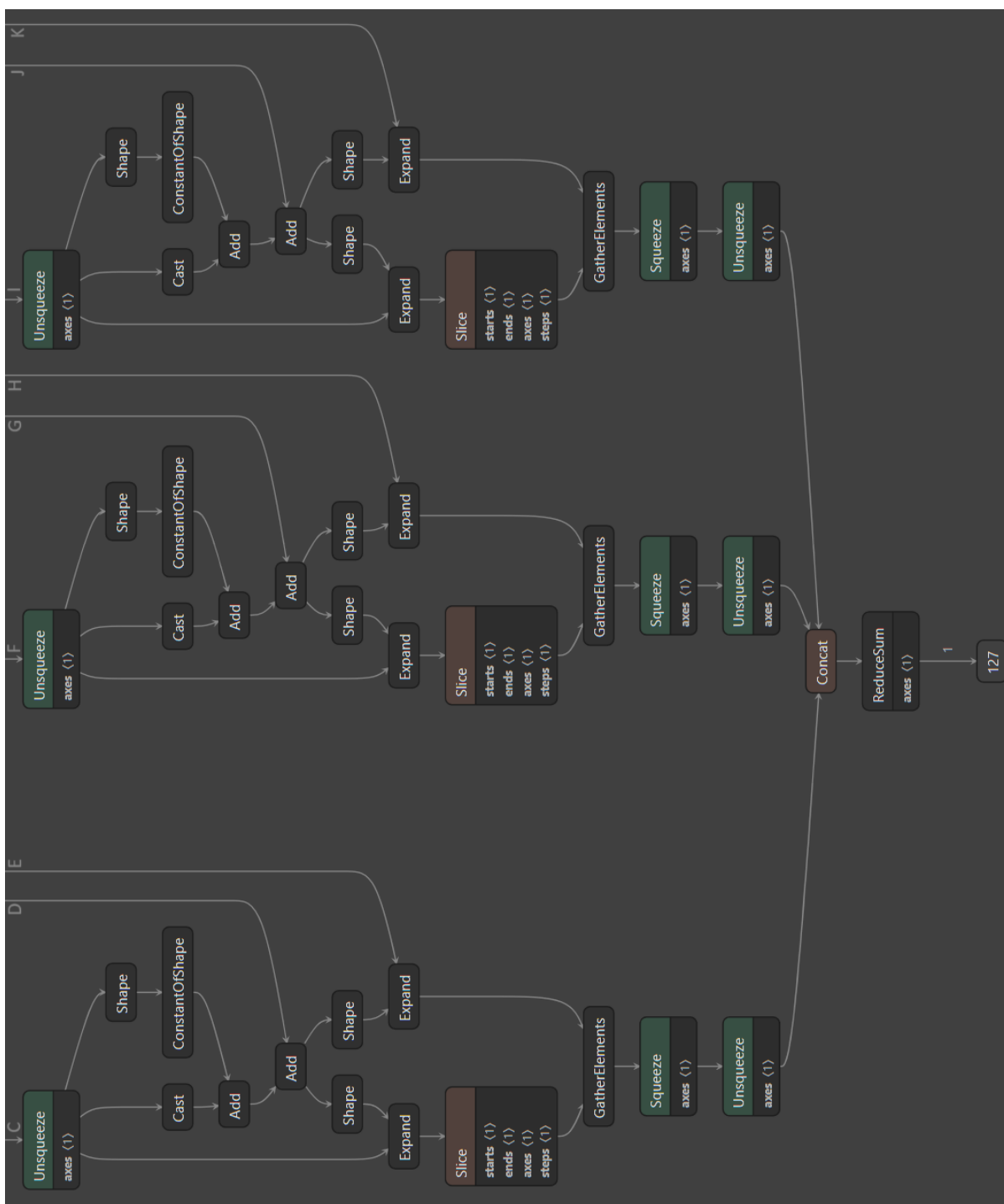
Obrázky 3.6 a 3.7 zobrazují exportovaný ONNX model finální verze ve dvou částech. Oproti první verzi (jednoduchý *actor-critic* s jedním diskrétním výstupem) je zde patrná vyšší komplexita, která odpovídá rozšířenému akčnímu prostoru: model již neřeší pouze volbu směru pohybu, ale samostatně rozhoduje také o rotaci, střelbě a volbě výbavy.

V modelu je zachována struktura s plně propojenými vrstvami („Gemm“) a nelinearitami („Tanh“), nicméně výstupní část je výrazně bohatší a je realizována více větvemi (*multi-head* výstupy). Tyto výstupy jsou typicky normalizovány pomocí „Softmax“ a převáděny na konkrétní akci pomocí „ArgMax“, přičemž podpůrné operace jako „Unsqueeze“, „Shape“, „Slice“ a „GatherElements“ slouží k úpravám tvarů tenzorů a k indexování jednotlivých složek akce při inferenci.

Z praktického hlediska tato architektura umožňuje, aby agent koordinoval více rozhodnutí v rámci jednoho kroku prostředí (pohyb, natočení a střelbu) a zároveň prováděl výběr výbavy mezi koly. To odpovídá pozorovanému zlepšení oproti druhé verzi, zejména v plynulosti chování, přesnosti střelby a udržování vhodné vzdálenosti.



Obrázek 3.6: Vzhled modelu z finální verze agenta 1/2



Obrázek 3.7: Vzhled modelu z finální verze agenta 2/2

3.2.7 Shrnutí všech verzí

Vyhodnocení jednotlivých verzí agenta ukazuje jasnou postupnou evoluci chování, komplexity i herní kompetence. Každá verze plnila specifickou roli v procesu vývoje a přispěla k ověření jiného aspektu návrhu.

První verze prokázala, že zvolený tréninkový framework a algoritmus PPO jsou schopny stabilně řešit jednoduché úlohy. Agent se v této fázi naučil spolehlivě navigovat v prostředí, avšak bez jakékoli herní interakce. Její význam spočíval především v technickém ověření celého systému, nikoli v herní použitelnosti.

Druhá verze představovala zásadní krok směrem k hernímu chování. Agent byl schopen aktivně vyhledávat protivníka, rotovat, střílet a vstupovat do bojových situací. Zároveň se však ukázalo, že samotné rozšíření akčního a pozorovacího prostoru není dostačující. Nevhodné vyvážení odměnové funkce vedlo k chování, které sice bylo krátkodobě efektivní, ale herně nepřírozené a nestabilní. Tato verze tak jasně odhalila limity návrhu bez důsledné kontroly optimalizačních cílů.

Finální verze agenta tyto nedostatky ve velké míře odstranila. Agent vykazuje konzistentní a čitelné chování, dokáže udržovat vhodnou vzdálenost, plynule se orientovat, útočit s vyšší přesností a adaptovat výbavu na základě předchozích kol. Z praktických testů vyplývá, že právě tato verze je pro hráče nejobtížnější a nejbližší zamýšlenému hernímu protivníkovi.

Současně se však ukázalo, že skutečná adaptace během samotného hraní je omezená. Ačkoliv agent disponuje mechanismy pro průběžné učení, nízký počet instancí v herním prostředí a omezení použitého frameworku NevarokML prakticky znemožňují smysluplnou online adaptaci. Agent tak například není schopen dynamicky upravovat optimální vzdálenost od protivníka ani adaptovat střelecké chování (např. volbu mezi rychlejší, méně přesnou střelbou a pomalejší, přesnější palbou) v reakci na konkrétní styl hráče. Tyto limity nejsou důsledkem návrhu agenta samotného, ale vyplývají z omezení použité infrastruktury a budou podrobně rozebrány v závěrečné kapitole práce.

Závěr

Hlavním cílem této diplomové práce bylo navrhnout autonomního agenta, který je schopen fungovat v dynamickém herním prostředí s využitím metod strojového učení a adaptovat své chování, včetně volby výbavy, s cílem efektivně čelit protivráči.

V úvodních kapitolách byly představeny existující přístupy k umělé inteligenci ve hrách a nejčastěji používané algoritmy a techniky. Mezi analyzovanými systémy se nacházely mimo jiné AlphaStar, OpenAI Five a Voyager; z metod pak zejména Proximal Policy Optimization (PPO), Deep Q-Network (DQN), Self-Play a Population-Based Training.

Následně byly popsány technologie použité při realizaci samotné hry a agenta, především herní engine Unreal Engine a plugin NevarokML. Součástí této části byl rovněž návrh herní logiky a architektury agenta.

Závěrečná část práce se zaměřila na implementaci jednotlivých herních modulů, jako jsou hráčská postava, zbraně, systém kol a mechanismy získávání výbavy, a především na návrh, trénování a vyhodnocení jednotlivých verzí agenta. Výsledný agent je schopen pohybu, rotace, útoku i změny výbavy a jeho chování je řízeno neuronovou sítí trénovanou metodami posilovaného učení.

První verze agenta byla záměrně jednoduchá a sloužila především k ověření práce s pluginem NevarokML a celým trénovacím frameworkem. Druhá verze již obsahovala kompletní bojovou logiku, avšak vykazovala nestabilní a místy nepřirozené chování, zejména v oblasti udržování vzdálenosti od protivníka. Finální verze agenta tyto nedostatky ve velké míře odstranila a nejlépe prokázala schopnost adaptivního chování v dynamickém prostředí. Agent je schopen aktivně pronásledovat protivníka, útočit a po jednotlivých kolech upravovat svou výbavu na základě předchozích střetů.

Navzdory dosaženým výsledkům se ukázalo, že skutečná adaptace agenta v reálném čase během samotného hraní je výrazně omezená. Tento stav je způsoben především omezeními pluginu NevarokML. V jeho současné podobě je například možné provozovat pouze jednoho trenéra na jedno dynamicky vytvářené herní prostředí, zatímco u statických trénovacích map tento problém nenastává, jelikož je možné přiřadit jednomu trenérovi více prostředí. Dále není podporována změna odměnové funkce, akčního prostoru ani vstupních pozorování v průběhu tréninku při zachování již naučeného modelu, tedy přístup známý jako Curriculum Learning, založený na postupném zvyšování obtížnosti úlohy. Z tohoto důvodu je nutné trénovat jeden komplexní model od počátku na plně složité úloze, což výrazně omezuje možnosti dosažení optimálního chování agenta.

Literatura

- [1] GOOGLE. *AlphaGo*. Online. GOOGLE. Google DeepMind. B. r. Dostupné z: <https://deepmind.google/technologies/alphago/>. [cit. 2025-05-10].
- [2] GeeksForGeeks. *Q-Learning*. Online. GeeksForGeeks. B. r. Dostupné z: <https://www.geeksforgeeks.org/machine-learning/q-learning-in-python/>. [cit. 2026-01-05].
- [3] Open AI. *ChatGPT*. Online. Dostupné z: <https://openai.com/index/chatgpt/>. [cit. 2026-01-05].
- [4] Valve. *Dota 2*. Online. Dostupné z: <https://www.dota2.com/home?l=czech>. [cit. 2026-01-05].
- [5] JADERBERG, Max; DALIBARD, Valentin a OSINDERO, Simon et al. *Population Based Training of Neural Networks*. Online, arxiv. USA: Cornell University, 2017. Dostupné z: <https://arxiv.org/abs/1711.09846>. [cit. 2025-05-10].
- [6] *MuJoCo*. Online. C2021. Dostupné z: <https://mujoco.org/>. [cit. 2025-05-10].
- [7] TEAM, OpenAI Five. *OpenAI Five defeats Dota 2 world champions*. Online. OpenAI. 2019. Dostupné z: <https://openai.com/index/openai-five-defeats-dota-2-world-champions/#timeline>. [cit. 2025-05-10].
- [8] *Proximal Policy Optimization*. Online. OpenAI. 2017. Dostupné z: <https://openai.com/index/openai-baselines-ppo/>. [cit. 2025-05-10].
- [9] SCHULMAN, John; LEVINE, Sergey; MORITZ, Philipp; JORDAN, Michael I. a ABBEEL, Pieter. *Trust Region Policy Optimization*. Online, arxiv. USA: Cornell University, 2015. Dostupné z: <http://arxiv.org/abs/1502.05477>. [cit. 2025-05-10].
- [10] SCHULMAN, John; WOLSKI, Filip; DHARIWAL, Prafulla; RADFORD, Alec a KLIMOV, Oleg. *Proximal Policy Optimization Algorithms*. Online, arxiv. USA: Cornell University, 2017. Dostupné z: <https://arxiv.org/abs/1707.06347>. [cit. 2025-05-10].
- [11] SCHULMAN, John; MORITZ, Philipp; LEVINE, Sergey; JORDAN, Michael a ABBEEL, Pieter. *High-Dimensional Continuous Control Using Generalized Advantage Estimation*. Online, arxiv. USA: Cornell University, 2018. Dostupné z: <https://arxiv.org/abs/1506.02438>. [cit. 2025-05-10].

- [12] VINYALS, Oriol; BABUSCHKIN, Igor; CHUNG, Junyoung et al. *AlphaStar: Mastering the real-time strategy game StarCraft II*. Online. Google DeepMind. B. r. Dostupné z: <https://deepmind.google/discover/blog/alphastar-mastering-the-real-time-strategy-game-starcraft-ii/>. [cit. 2025-05-10].
- [13] WANG, Guanzhi; XIE, Yuqi a JIANG, Yunfan et al. *Voyager: An Open-Ended Embodied Agent with Large Language Models*. Online. MineDojo. B. r. Dostupné z: <https://voyager.minedojo.org/>. [cit. 2025-05-10].
- [14] WATKINS, Christopher J. C. H. a DAYAN, Peter. Q-learning. Online. *Machine Learning*. 1992, roč. 8, č. 11, s. 279—292. Dostupné z: <https://doi.org/10.1007/BF00992698>. [cit. 2025-05-10].
- [15] IBM. *Deep Blue (chess computer)*. Online. Dostupné z: <https://www.ibm.com/history/deep-blue>. [cit. 2026-01-05].
- [16] Blizzard. *StarCraft II*. Dostupné z: <https://starcraft2.blizzard.com/en-us/>. [cit. 2026-01-05].
- [17] ZHANG, Ruize; XU, Zelai a MA, Chengdong et al. *A Survey on Self-play Methods in Reinforcement Learning*. Online, arxiv. USA: Cornell University, 2024. Dostupné z: <https://arxiv.org/abs/2408.01072>. [cit. 2025-05-10].
- [18] *Unreal Engine* [online]. USA: Epic Games, 2025 [cit. 2025-07-31]. Dostupné z: www.unrealengine.com
- [19] *NevarokML* [online]. China: Kyrylo Mishakin, 2025 [cit. 2025-07-31]. Dostupné z: <https://nevarok.com/nevarok-ml/>
- [20] *ONNX* [online]. USA: Microsoft, AMD et., 2025 [cit. 2025-07-31]. Dostupné z: www.onnx.ai
- [21] *Neural Network Engine* [online]. USA: Epic Games, 2025 [cit. 2025-07-31]. Dostupné z: <https://dev.epicgames.com/documentation/en-us/unreal-engine/neural-network-engine-overview-in-unreal-engine>
- [22] SUTTON, Richard S. a BARTO, Andrew G. *Reinforcement learning: an introduction*. Second edition. Adaptive computation and machine learning. Cambridge, Massachusetts: The MIT Press, [2018]. ISBN 978-0-262-03924-6.
- [23] MNIH, Volodymyr; KAVUKCUOGLU, Koray; SILVER, David et al. *Playing Atari with Deep Reinforcement Learning*. Online, arxiv. USA: Cornell University, 2013. Dostupné z: <https://arxiv.org/abs/1312.5602>. [cit. 2025-07-31].
- [24] MNIH, Volodymyr; KAVUKCUOGLU, Koray; SILVER, David et al. Human-level control through deep reinforcement learning. Online. *Nature*. 2015, č. 518, s. 529—533. Dostupné z: <https://doi.org/10.1038/nature14236>. [cit. 2025-07-31].

- [25] TEAM, AlphaStar. *AlphaStar: Mastering the real-time strategy game StarCraft II*. 2019. Dostupné z: <https://deepmind.google/discover/blog/alphastar-mastering-the-real-time-strategy-game-starcraft-ii/>. [cit. 2025-07-31].
- [26] TEAM, OpenAI Five. *OpenAI Five*. 2018. Dostupné z: <https://openai.com/index/openai-five/>. [cit. 2025-07-31].
- [27] WANG, Guanzhi; XIE, Yuqi a JIANG, Yunfan et al. *Voyager: An Open-Ended Embodied Agent with Large Language Models*. 2023. Dostupné z: <https://voyager.minedojo.org/>. [cit. 2025-07-31].
- [28] PATHMAKUMAR, Thejus et al. *A Reinforcement Learning Based Dirt-Exploration for Cleaning-Auditing Robot*. 2021. Dostupné z: https://www.researchgate.net/figure/The-reinforcement-learning-architecture-based-on-Proximal-Policy-Optimization-fig3_357019705. [cit. 2025-07-31].
- [29] STROUSE, DJ et al. *Collaborating with Humans without Human Data*. Online, arxiv. USA: Cornell University, 2021. Dostupné z: <https://arxiv.org/abs/2110.08176>. [cit. 2025-07-31].
- [30] JADERBERG, Max. *Population based training of neural networks*. 2017. Dostupné z: <https://deepmind.google/discover/blog/population-based-training-of-neural-networks/>. [cit. 2025-07-31].
- [31] ELSAYED, Medhat et al. *AI-enabled Future Wireless Networks: Challenges, Opportunities and Open Issues*. 2021. Dostupné z: https://www.researchgate.net/publication/349913716_AI-enabled_Future_Wireless_Networks_Challenges_Opportunities_and_Open_Issues. [cit. 2025-07-31].
- [32] JOCHEC, Martin. *Adaptace autonomního agenta na chování uživatele*. Bakalářská práce. České vysoké učení technické v Děčíně: České vysoké učení technické v Praze, 2023. Dostupné také z: <http://hdl.handle.net/10467/111458>.
- [33] JOCHEC, Martin. *Strojové učení pro adaptaci autonomních agentů*. Výzkumný úkol. České vysoké učení technické v Praze: České vysoké učení technické v Praze, 2024.

Příloha A

Jak zapnout program

1. Nejprve si stáhněte soubor přiložený k diplomové práci z OneDrivu, který obsahuje samotnou hru.
2. V souboru naleznete .exe pod jménem RoboHell, tímto hru spustíte.
3. Tímto je vše připraveno. Projekt by nyní měl být plně funkční.